



MÜHENDİSLİK FAKÜLTESİ
YAZILIM MÜHENDİSLİĞİ BÖLÜMÜ

YAZILIM MÜHENDİSLİĞİ DERSİ
SCRUM TABANLI TAKIM PROJESİ RAPORU

Ders Adı: Yazılım Projesi Geliştirme

Proje Adı: DermaSmart İçerik Analizi İle Kişiselleştirilmiş
Cilt Bakımı Uygulaması

Takım Adı: MethodX

Takım Üyeleri:

Hayrunnida ŞAHİN (Product Owner)
Şevval ARSLAN (Scrum Master)
Zeynep EKİNCİ (Developer)
Senanur TURUNÇ (Developer)
Senanur KURŞUN (Developer)
Ayşenur KÜÇÜKASLAN (Developer)

Öğretim Elemanı: Yavuz İNCE

Teslim Tarihi:

26.06.2026

Özet

DermaSmart projesi, bilinçsiz kozmetik tüketiminin ve birbiriyle uyuşmayan aktif içeriklerin (örn. Retinol ve C Vitamini) bir arada kullanılması sonucu oluşan cilt bariyeri hasarlarını önlemek amacıyla geliştirilmiş bir mobil asistan sistemidir. Projenin amacı, son kullanıcılara dermatolojik standartlara uygun, içerik çakışması barındırmayan kişiselleştirilmiş cilt bakım rutinleri sunmaktır. Proje metodolojisi olarak Çevik (**Agile**) felsefe ve **Scrum** çerçevesi benimsenmiştir. Altı kişilik geliştirme ekibiyle yürütülen süreçte kullanılan yöntemler sırasıyla; ham veri setleri üzerinde veri ön işleme (**Data Preprocessing**), kural tabanlı uyuşmazlık motoru tasarımı ve N-Katmanlı (**N-Tier**) mimari kurgusudur. Sistem; sunucu tarafında ASP.NET Core tabanlı bir web servisi (**Web Application / API**), istemci tarafında ise Flutter tabanlı bir mobil uygulama (**Mobile Application**) olarak inşa edilmiştir. Altyapı bulut mimarisine uyumlu (*cloud-ready*) tasarlanmış; ancak sıfır gecikme (*zero latency*) ve yüksek güvenlik hedefleri doğrultusunda proje sunumu yerel ağ (*Local Intranet*) üzerinde izole edilerek API haberleşmesi HTTP protokolüyle gerçekleştirilmiştir. Gerçekleştirilen sistem testleri sonucunda; MVP kapsamındaki dinamik katalog filtreleme ve AM/PM rutin oluşturma fonksiyonlarının %100 kararlılıkla çalıştığı doğrulanmıştır. Ortaya konan proje çıktıları; doğrulanmış bir ilişkisel veri tabanı, dinamik durum yönetimi barındıran (GridView/ListView) bir mobil ürün ve teknik **dokümantasyondur**. Proje, hem zamansal hem de teknik kısıtları disiplinli bir Backlog yönetimiyle aşarak başarıyla teslim edilmiştir

1. Giriş

1.1 Problem Tanımı

Kozmetik sektöründeki bilgi kirliliği ve agresif pazarlama stratejileri, tüketicileri kendi cilt tiplerini tanımadan rastgele ürün satın almaya itmektedir. Bu durumun yol açtığı en kritik mühendislik ve biyokimyasal problem; birbiriyle reaksiyona giren veya pH değerleri uyuşmayan aktif içeriklerin (Örn: Retinol ile C Vitamini veya Peeling Asitleri) aynı rutinde kullanılması sonucu cilt bariyerinin tahrip edilmesidir. Kullanıcılar, edindikleri kozmetiklerin hangi sıralamayla, günün hangi diliminde kullanılacağını bilmemekte ve sürdürülebilir bir takip mekanizması kuramamaktadır.

1.2 Proje Amacı

DermaSmart projesinin temel amacı; kullanıcının dermatolojik profilini dijitalleştiren, ürünlerin içerik (ingredient) listelerini algoritmik bir denetimden geçirerek olası aktif madde uyuşmazlıklarını bloklayan ve kullanıcıya güvenli sabah/akşam rutinleri oluşturan bir mobil yazılım sistemini uçtan uca tasarlamak, kodlamak ve doğrulamaktır.

1.3 Kapsam

Projenin Minimum Çalışır Ürün (MVP) kapsamı şu üç temel modülü barındıracak şekilde sınırlandırılmış ve teslim edilmiştir:

1. **Cilt Analiz Modülü:** Kullanıcının cilt tipini (Yağlı, Kuru, Karma, Hassas) ve ana problemlerini saptayan algoritmik anket akışı.

2. **Dinamik Ürün Kataloğu:** Veri ön işlemeden geçirilmiş ürün veri tabanı üzerinde **isim ve marka bazlı** metinsel arama yapabilen; yatay düzlemde konumlandırılmış seçim etiketleri (*Choice Chips*) aracılığıyla yalnızca "Cilt Tipine" göre (*Tümü, Yağlı Cilt, Kuru Cilt, Karma Cilt*) anlık durum filtrelemesi uygulayan ve eşleşen sonuçları; cilt tipi etiketleri, ürün adı, marka ve fiyat bilgisiyle birlikte iki kolonlu ızgara (GridView) yapısında render eden listeleme arayüzü.
3. **Rutin Planlayıcı:** Seçilen ürünleri "Çakışma Kural Motoru" denetiminden geçirerek Sabah (AM) ve Akşam (PM) rutinlerine atayan zamanlama aracı.

1.4 Paydaşlar

- **Son Kullanıcılar:** Güvenli cilt bakım rutini oluşturmak ve ürün içeriği doğrulamak isteyen mobil cihaz sahipleri.
- **Müşteriler:** Sisteme veri veya sponsorluk entegrasyonu sağlayabilecek dermokozmetik markaları ve dermatoloji klinikleri.
- **Geliştirme Ekibi:** Product Owner, Scrum Master ve Geliştiricilerden oluşan 6 kişilik mühendislik takımı.
- **Sistem Yöneticileri:** Projenin akademik bir prototip olması sebebiyle ayrı bir operasyon (Ops) personeli bulunmamaktadır.

2. Gereksinim Analizi

2.1 Kullanıcı Gereksinimleri

- **Gereksinim:** Kullanıcı, karmaşık dermatolojik terminolojiyle boğulmadan kendi cilt tipini ve temel endişelerini saptayabilmelidir.
 - **Gerçekleşme:** Adım tabanlı (*Stepper*) bir anket akışı geliştirilmiştir.
- **Gereksinim:** Kullanıcı, geniş kozmetik havuzu içerisinde bilgi kirliliğine maruz kalmadan, yalnızca kendi cilt profiline uygun ürünleri sezgisel bir biçimde görüntüleyebilmelidir.
 - **Gerçekleşme:** Katalog arayüzüne; analiz anketinden dönen "Cilt Tipi" verisini parametre alan ve yatay seçim etiketleri (*Choice Chips*) üzerinden anlık durum (*State*) filtrelemesi uygulayan dinamik bir modül entegre edilmiştir.
- **Gereksinim:** Kullanıcı, seçtiği ürünlerle sabah (AM) ve akşam (PM) zaman dilimleri için günlük rutinler oluşturabilmeli ve bu işlem sırasında sağlığını tehdit edebilecek kimyasal içerik çakışmalarına karşı sistem tarafından uyarılmalıdır.
 - **Gerçekleşme:** Ürün detay sayfalarına "Rutine Ekle" modülü konumlandırılmış ve ekleme isteği atıldığı an arka planda çalışan kural tabanlı çakışma denetleyicisi (Rule-Based Engine) devreye alınarak, uyumsuzluk anında işlemi durduran bir güvenlik bariyeri **geliştirilmiştir**.
- **Gereksinim:** Kullanıcı, cilt bakım rutini sürdürme konusundaki istikrarını takip edebilmeli ve günlük görevlerini tamamladıkça motivasyonunu artıracak istatistiksel geri bildirimler alabilmelidir.

- **Gerçekleşme:** Kullanıcı bağlılığını (retention) ölçmek ve oyunlaştırma (gamification) dinamiklerini kullanmak amacıyla; günlük seriyi (*streak*), aylık başarı yüzdesini ve takvim tabanlı tamamlanma durumunu görselleştiren bağımsız bir "İlerleme" ekranı **tasarlanmıştır**.

2.2 Fonksiyonel Gereksinimler

- **Veri Ön İşleme ve İndeksleme (Data Preprocessing):** Sistemin kural tabanlı çakışma algoritmalarına altlık oluşturmaya amacıyla ham veri seti üzerinde temizleme işlemleri gerçekleştirilmiştir. Eksik ve tekrar eden kayıtlar silinmiş; kozmetik verileri *Ürün Adı/Markası*, *Aktif/Yardımcı İçerikler* ve *Cilt Tipine Uygunluk* matrisi formatında normalize edilerek ilişkisel veri tabanına indekslenmiştir.
- **Dinamik Önyüz Listeleme:** Flutter katmanındaki ürün kataloğu, cihaz belleğini optimize etmek amacıyla statik yapılar yerine dinamik durum yönetimi (*State Management*) ile kodlanmıştır. İlgili modül, anketten elde edilen "Cilt Tipi" parametresini ve yatay seçim etiketlerindeki (*Choice Chips*) anlık tercihleri baz alarak veri tabanını süzmekte; eşleşen sonuç kümesini GridView.builder yapısı üzerinden asenkron olarak 2'li kolon matrisi halinde render etmektedir.
- **Çakışma Denetleyici Servisi (Conflict Resolution Engine):** Kullanıcı bir ürünü rutinine eklemek istediğinde, backend servisine asenkron bir HTTP isteği iletilmektedir. İlgili API servisi, seçilen yeni ürünün aktif bileşenlerini (ingredients), kullanıcının mevcut rutinindeki diğer aktif bileşenlerle çapraz kontrole (*cross-check*) sokmaktadır. Kimyasal veya pH bazlı bir çakışma kuralı ihlali tespit edildiği an, veri tabanına yazma işlemi sunucu tarafında (backend) iptal edilerek istemci arayüzüne (*client*) uyarı fırlatılmaktadır.

2.3 Fonksiyonel Olmayan Gereksinimler

- **Güvenlik:** Kullanıcıların anket cevapları ve kişisel cilt profilleri veri tabanında anonimleştirilerek saklanmaktadır. Sistemin nihai mimarisinde, mobil istemci ile Web API arasındaki tüm veri trafiğinin uçtan uca HTTPS protokolü ile şifrelenmesi tasarlanmıştır. Oturum yönetimi ise aktif olarak JWT (JSON Web Token) standartları kullanılarak yetkilendirilmektedir.

(Not: Proje şu aşamada yerel geliştirme ortamında test edildiğinden ağ iletişimi geçici olarak HTTP üzerinden sağlanmaktadır; ancak üretim ortamına (production) geçişte SSL/TLS sertifikasyonu ile HTTPS yapısı uygulanması planlanmaktadır.)

- **Performans:** Mobil uygulamanın cihaz belleği (RAM) tüketimini minimize etmek amacıyla, çoklu verilerin listelendiği katalog ekranlarında tembel yükleme (*lazy loading*) yapısı kurulmuştur. Bu sayede veri tabanı sorgu yükü hafifletilmiş ve mobil arayüzün (UI) render akıcılığı optimize edilmiştir.

- **Kullanılabilirlik:** Ekran arayüzleri, son kullanıcının bilişsel yükünü (*cognitive load*) en aza indirecek minimalist bir UI/UX yaklaşımıyla tasarlanmıştır. Filtreleme, arama ve rutine ekleme adımları, mobil erişilebilirlik standartlarına uygun ve sezgisel (*heuristic*) bir yapıya kavuşturulmuştur.
- **Sürdürülebilirlik:** Yazılım mimarisi prensipleri doğrultusunda, API (iş mantığı) katmanını ile mobil istemci arayüzü tamamen birbirinden ayırık (*decoupled*) olarak kurgulanmıştır. Bu modüler altyapı sayesinde; veri tabanına yeni bir kozmetik ürünü veya dermatolojik çakışma kuralı eklendiğinde, kullanıcı tarafında bir market güncellemesine gerek duyulmaksızın sistemin dinamik olarak beslenmesi sağlanmıştır.

2.4 Kısıtlar

- **Zaman Kısıtı:** Bir akademik dönemle sınırlı olan geliştirme süresi; ürün vizyonu dışındaki tüm tali taleplerin Product Owner tarafından Backlog üzerinde dondurulması (*Scope Management*) ve eforun tamamen çekirdek MVP adımlarına yönlendirilmesiyle başarılı bir şekilde yönetilmiştir.
- **İnsan Kaynağı Kısıtı:** Altı kişilik geliştirme ekibinin eşzamanlı (senkron) toplanamaması problemi; düzenli Daily Scrum ve katı bir Git-Flow dallanma modeli, **ClickUp** üzerinden asenkron görev (*Task*) takibi ve kod birleştirme (*merge*) öncesi zorunlu *Code Review* pratikleri işletilerek çözülmüştür.
- **Teknoloji Kısıtı:** İstemci tarafındaki Flutter mimarisi ile C# tabanlı Web API arasındaki veri transferinde karşılaşılan model uyumsuzlukları, projenin başlangıç fazında ortak bir JSON DTO (*Data Transfer Object*) şeması dokümanite edilerek ve katı veri kontratları oluşturularak aşılmıştır.
- **Bütçe Kısıtı:** Projenin akademik doğası gereği karşılaşılan sunucu ve veri tabanı barındırma maliyetleri; bulut servis sağlayıcılarının sunduğu ücretsiz geliştirici katmanı (*Free-Tier*) limitleri dâhilinde kalınacak şekilde, optimize edilmiş SQL sorguları yazılarak ve veri tabanı yükü hafifletilerek sıfır maliyetle tamamlanmıştır.

3. Mühendislik Tasarımı

3.1 Alternatif Çözüm Yaklaşımları

Sistem geliştirilmeye başlanmadan önce, cilt bakımı öneri ve çakışma kontrol işlevlerinin nerede çalıştırılacağı ve verilerin nasıl yönetileceği üzerine üç alternatif mimari tasarım değerlendirilmiştir:

- **Alternatif Tasarım 1: İstemci Tabanlı (Client-Side) ve Yerel Veri Depolama Tasarımı** Bu tasarımda, Kaggle'dan elde edilip ön işlemden geçirilerek birleştirilen 4 farklı cilt bakım ürün veri seti (CSV) ve içerik çakışma kuralları mobil uygulamanın (Flutter) yerel varlıkları (Assets) içerisine gömülür. Cilt analiz anketi çözüldüğünde, algoritma tamamen cihaz işlemcisi üzerinde çalışır. Kullanıcının cilt profili ve oluşturulan rutinler cihazın yerel hafızasında (SQLite veya Hive veritabanı ile) saklanır. Herhangi bir API isteği veya internet bağlantısı gerekmeksizin tüm süreç cihaz içinde başlar ve biter. Ancak, veri tabanına yeni bir ürün eklenmesi veya çakışma kuralının güncellenmesi

durumunda tüm mobil uygulamanın derlenip uygulama mağazalarında güncellenmesi zorunludur.

- Alternatif Tasarım 2: Sunucu Tabanlı Katmanlı Mimari Tasarımı (Seçilen Çözüm) Mobil istemci (Flutter) sadece arayüz sunumu ve kullanıcı girdilerini almakla sorumludur. Algoritmanın kalbini oluşturan çakışma motoru (Conflict Engine) ve veri seti entegrasyonu (Celestia Service) backend tarafında (ASP.NET Core Web API) konumlandırılmıştır. Veriler sunucudaki SQLite ilişkisel veritabanında yönetilir. İstemci, kullanıcının profil verilerini JWT (JSON Web Token) kimlik doğrulamasıyla /api/routine/generate endpoint'ine gönderir. Sunucu, veri setini ve çakışma kurallarını işletip sıralı rutini üretir, veritabanına kaydeder ve istemciye JSON olarak döner. Ürün güncellemeleri ve kural değişiklikleri sunucu tarafında anında yapılır, veri gizliliği en üst düzeyde korunur.
- Alternatif Tasarım 3: Mikroservis (Microservices) Mimarisi Tasarımı Sistemin tüm modülleri bağımsız servislere bölünür. Kimlik Doğrulama Servisi (Auth Service), Rutin Oluşturma Servisi (Routine Service), Ürün Kataloğu Servisi (Product Service) ve Takip Servisi (Tracking Service) ayrı sunucularda bağımsız docker konteynerleri olarak çalıştırılır. Haberleşme bir API Gateway üzerinden sağlanır. İstemci istekleri API Gateway'e gelir, gateway istekleri ilgili mikroservise yönlendirir. Her servisin kendine ait bağımsız bir veritabanı bulunur. Ancak küçük bir geliştirici ekibi ve prototip aşamasındaki bir proje için sistem karmaşıklığı, test edilebilirliği ve dağıtım (deployment) maliyeti aşırı yüksektir.

3.2 Seçilen Çözümün Gerekçesi

Değerlendirilen alternatif tasarımlar; geliştirme maliyeti, veri güncellenebilirliği, güvenlik ve ölçeklenebilirlik kriterlerine göre kıyaslanarak puanlanmıştır (Puanlama derecesi: 1-En Kötü, 5-En İyi).

Kriterler	Alternatif Tasarım 1 (İstemci Tabanlı)	Alternatif Tasarım 2 (Seçilen - Katmanlı API)	Alternatif Tasarım 3 (Mikroservis)
Geliştirme Kolaylığı & Süresi	4	5	2
Veri Güncellenebilirliği (Uygulama Güncellemen)	1	5	5
Veri Güvenliği & Gizliliği (JWT)	2	5	5
Sunucu Maliyeti & Donanım Gereksinimi	5	4	1
Toplam Skor	12/20	19/20	13/20

Seçim Gerekçesi: Alternatif Tasarım 1; veri setinde yapılacak en ufak bir içerik güncellemesinde (örneğin yeni bir ürün veya çakışma kuralı eklendiğinde) tüm kullanıcılara uygulama

mağazalarından güncelleme zorunluluğu getirdiği için elenmiştir. Alternatif Tasarım 3 ise bu ölçekteki bir proje için gereksiz sistem karmaşıklığı (over-engineering) yaratacağı ve sunucu donanım maliyetlerini aşırı artıracacağı için tercih edilmemiştir.

Seçilen Alternatif Tasarım 2 (Katmanlı API Mimarisi); hem veri setlerinin sunucudan anlık güncellenebilmesini sağlamış, hem JWT ile kullanıcı verilerini güvence altına almış, hem de geliştirme maliyetini ve süresini en optimal düzeyde tutarak projeyi başarıya ulaştırmıştır.

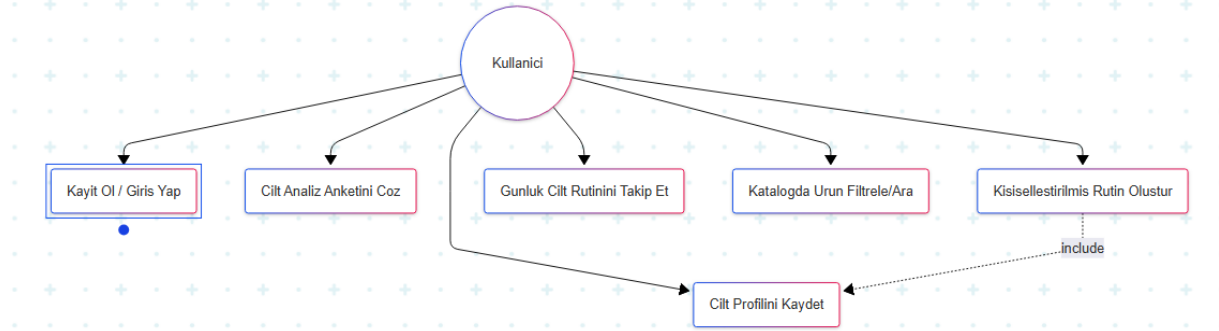
3.3 Sistem Mimarisi

DermaSmart sistemi, istemci, uygulama sunucusu ve veritabanı bileşenlerinin birbirinden bağımsız çalıştığı ve katı kurallarla haberleştiği 3 Katmanlı Mimari (3-Tier Layered Architecture) ve backend tarafında MVC (Model-View-Controller) tasarım deseni kullanılarak inşa edilmiştir. Sistem mimarisinin katmanları şu şekildedir:

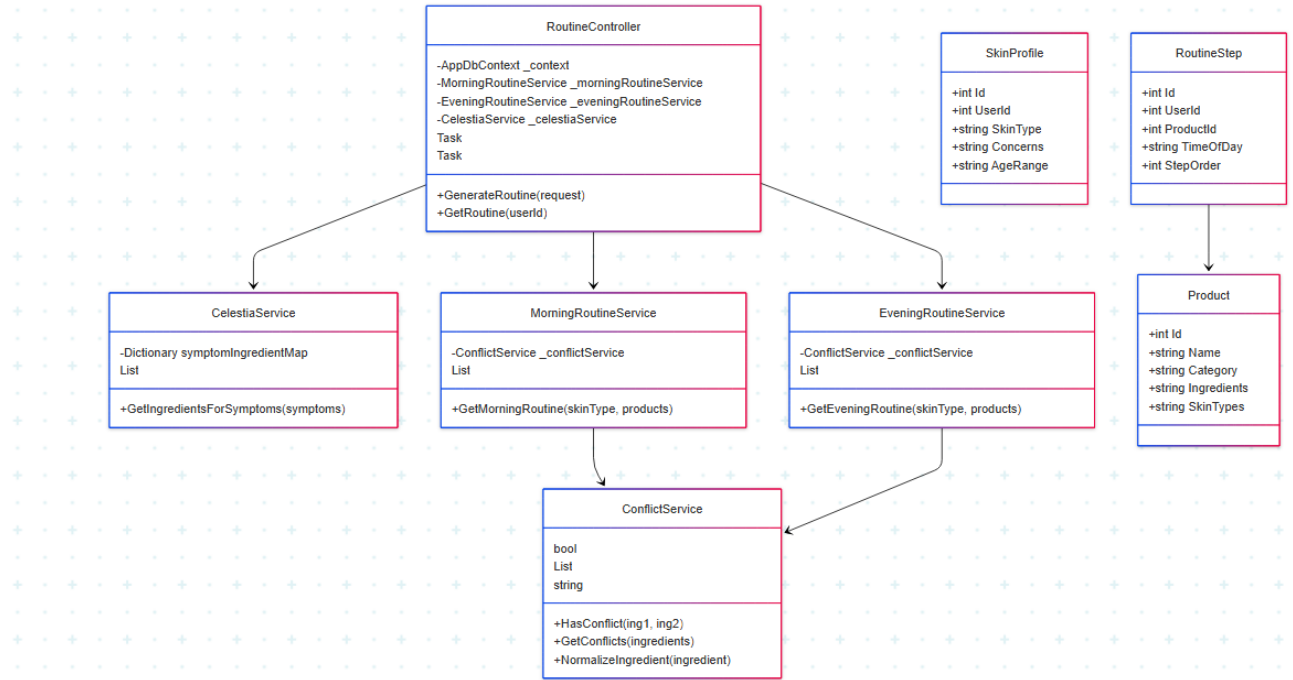
1. **Sunum Katmanı (Presentation Layer - Flutter İstemcisi):** Sistemin kullanıcıyla temas eden ön yüzüdür. Flutter framework'ü kullanılarak modüler yapıda geliştirilmiştir. Kullanıcıdan alınan anket girdilerini (cilt tipi, yaş, endişeler) toplar, HTTP REST API protokolü (http kütüphanesi) üzerinden backend sunucusuna iletir ve dönen JSON verilerini görsel arayüze (Dashboard, Rutin Kartları, Katalog) yansıtır.
2. **Uygulama ve İş Mantığı Katmanı (Application & Business Logic Layer - ASP.NET Core Web API):** Sistemin kalbini oluşturan, C# dili ile yazılmış API katmanıdır. Gelen istekleri karşılayan ve işleyen Controllers ve Services olarak kendi içinde iki alt katmana ayrılır:
 - Controllers (Denetleyiciler - MVC'nin C'si): RoutineController, SkinProfileController ve ProductsController sınıfları frontend'den gelen HTTPS isteklerini (endpoints) karşılar, yetkilendirme (JWT) ve girdi doğrulamalarını (DTOs) yapar.
 - Services (İş Servisleri): Algoritmaların çalıştığı bölümdür. CelestiaService veri setini yönetirken, ConflictService çakışmaları hesaplar. Morning/EveningRoutineService ise ürünleri süzüp sıralar.
3. **Veri Erişim Katmanı (Data Access Layer - EF Core & SQLite):** Verilerin kalıcı olarak saklandığı ve yönetildiği katmandır. ORM (Object-Relational Mapping) olarak Entity Framework Core kullanılmıştır. C# üzerinde tanımlanan Modeller (Models - MVC'nin M'si) (Product, SkinProfile, RoutineStep), EF Core vasıtasıyla ilişkisel SQLite veritabanındaki tablolara eşlenir. Veritabanı sorguları ve kayıt işlemleri bu katmanda güvenli bir şekilde yürütülür.

3.4 UML Diyagramları

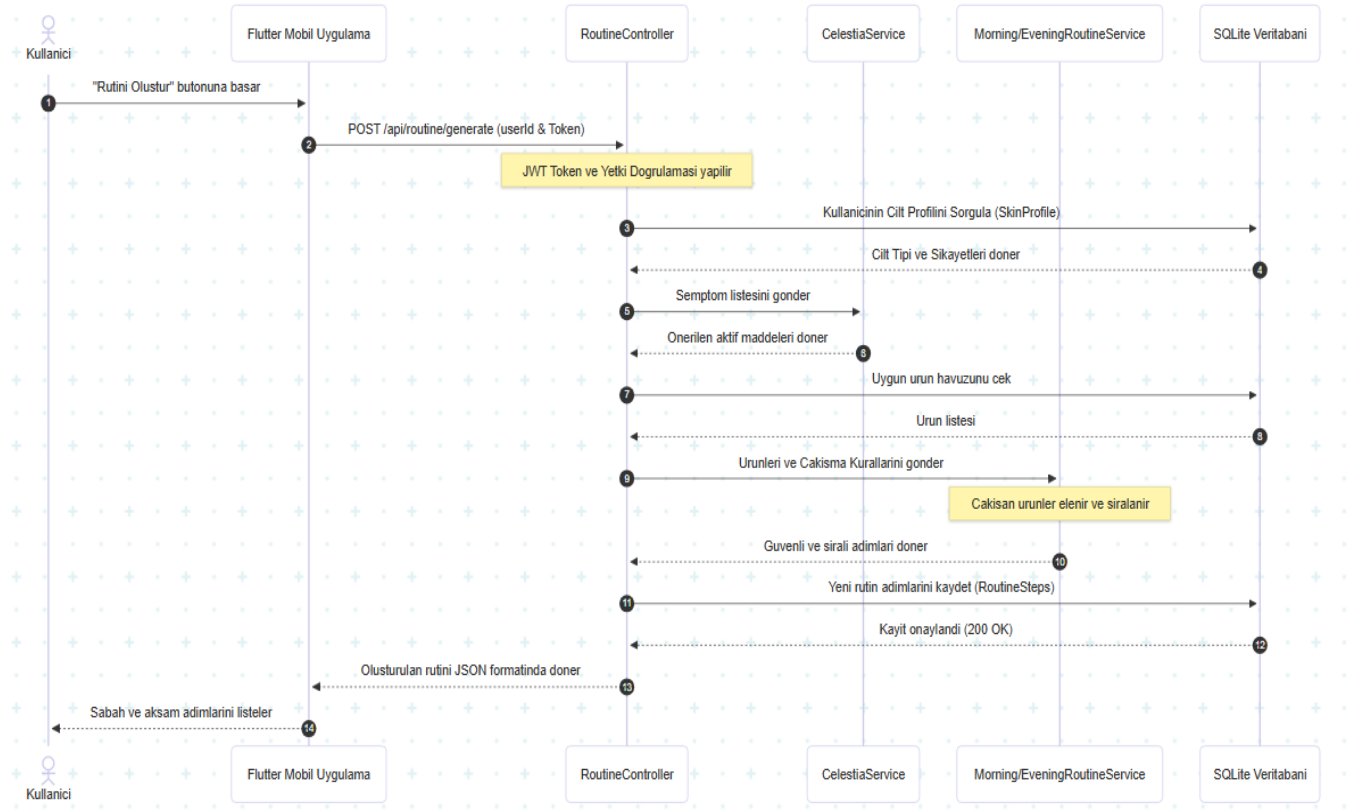
Kullanım Senaryosu (Use Case) Diyagramı



Sınıf (Class) Diyagramı



Ardışıl İşlem (Sequence) Diyagramı



Aktivite (Activity) Diyagramı



4. Scrum Süreci Yönetimi

Projemizin çevik (agile) geliştirme sürecinde, 6 kişilik çapraz fonksiyonlu (cross-functional) ekibimiz Scrum metodolojisine uygun olarak organize edilmiştir. Projenin başarılı bir şekilde canlıya alınabilmesi (deployment) için her bir ekip üyesinin üstlendiği ana roller ve temel sorumluluk alanları aşağıdaki tabloda özetlenmiştir:

4.1 Takım Organizasyonu

Üye	Sorumluluk	Rol
Hayrunnida Şahin	Ürün vizyonu ve MVP kapsamının yönetimi, kullanıcı hikayelerinin oluşturulması, veri setlerinin temizlenmesi, katalog/ürün ekranlarının UI/UX tasarımı ve dinamik liste geliştirmeleri.	Product Owner (Ek Sorumluluk: Frontend Destek Geliştirici)
Şevval Arslan	Scrum yönetilmesi (Daily Scrum, Sprint Planning, Review ve Retrospective), takımın ilerlemesinin önündeki engellerin kaldırılması, ClickUp üzerinden takımın takibi. Temel proje mimari kurulması, GitHub yönetimi ve API entegrasyonlarında kod seviyesinde destek.	Scrum Master (Ek Sorumluluk: Backend & Frontend Entegrasyonu Destek Geliştiricisi)
Zeynep Ekinci	ASP.NET Core iskeletinin ve SQLite/Entity Framework veritabanı mimarisinin kurulması, JWT kimlik doğrulama (Auth), güvenli form API'leri, performans (Rate Limit) ve Swagger dokümantasyonunun sağlanması.	Developer, Backend Mimarisi, Güvenlik ve API Geliştiricisi
Senanur Turunç	Kural tabanlı içerik çıkarma motorunun ve eşleştirme algoritmasının yazılması, dinamik rutin öneri/filtreleme API'lerinin geliştirilmesi, Quiz UI kodlaması ve UAT algoritma testlerinin yapılması.	Developer, API İş Mantığı (Business Logic) ve Algoritma Geliştiricisi
Senanur Kurşun	Tüm uygulama ekranlarının (Dashboard, Profil vb.) responsive tasarlanması, canlı API entegrasyonlarının	Developer, Frontend (UI/UX) ve API Entegrasyon Geliştiricisi

	yapılması, state yönetimi, form güvenliği (Validation) ve arayüz hata yönetiminin sağlanması.	
Ayşenur Küçükaslan	Kozmetik veri setlerinin analizine destek, SQL şema tasarımının yapılması, algoritma ve API'lerin fonksiyonel testleri (Postman), uçtan uca (E2E) mobil arayüz testleri ve hata raporlaması.	Developer, Kalite Güvence (QA), Veri Analizi ve Test Uzmanı

4.2 Product Backlog

Önemli kullanıcı hikayeleri:

- [US-01] Kullanıcı sisteme kayıt olabilmeli ve giriş yapabilmeli.
- [US-02] Kullanıcı cilt tipi anketini doldurabilmeli (kuru/yağlı/karma/hassas).
- [US-03] Kullanıcı cilt problemlerini çoklu seçebilmeli (akne, leke, kuruluk, hassasiyet vb.).
- [US-04] Sistem cilt profiline göre sabah rutini oluşturabilmeli.
- [US-05] Sistem cilt profiline göre akşam rutini oluşturabilmeli.
- [US-06] Rutin adımları doğru uygulama sırasıyla listelenmeli.
- [US-07] İçerik çakışmaları önlenmeli (niasinamid+C vit., glikolik asit+retinol vb.).
- [US-08] Kullanıcı günlük rutinini "tamamlandı" olarak işaretleyebilmeli.
- [US-09] Kullanıcı takvim üzerinden geçmiş rutin uyumunu görebilmeli.
- [US-10] Kullanıcı ürün kataloğunda cilt tipine göre filtreleme yapabilmeli.
- [US-11] Sistem kullanıcı verilerini şifreli ve güvenli saklayabilmeli.
- [US-12] Uygulama iOS ve Android'de sorunsuz çalışabilmeli.

4.3 Sprint Planları

Sprint süreçlerine ait iş paketlerinin, geliştirici atamalarının ve efor tahminlemelerinin dijital takibine [ClickUp Public Board](#) üzerinden ulaşılabilir.

Sprint 1

- **Sprint Amacı:** Projenin teknik iskeletini oluşturmak, versiyon kontrol sistemlerini kurmak ve temel kullanıcı veri güvenliğini sağlamak.
- **Planlanan Görevler:** Teknoloji stack kararlaştırılır (Flutter, ASP.NET Core, SQLite, Git repo kurulur, branch stratejisi belirlenir, Flutter proje kurulumu, navigasyon yapısı, ekran iskelet tasarımı yapılır. Kullanıcı hikayelerinden; [US-01] Kullanıcı sisteme kayıt olabilmeli ve giriş yapabilmeli ile [US-11] Sistem kullanıcı verilerini şifreli ve güvenli saklayabilmeli görevleri planlanmıştır.
- **Gerçekleşen Görevler:** Planlanan tüm görevler "COMPLETED" (Tamamlandı) statüsüne çekilerek sprint başarıyla kapatılmıştır.

Sprint 2

- **Sprint Amacı:** Kural motorunu inşa etmek, kayıt/onboarding arayüzlerini tamamlamak ve uçtan uca (E2E) sistem testlerini başlatmak.
- **Planlanan Görevler:** Kayıt, giriş ve onboarding ekranlarının Flutter'da geliştirilmesi, E2E (Uçtan Uca) Test: Mobil arayüzden Backend algoritmalarına kadar tüm sistemin testi, Sprint review hazırlığı ve son onaylar. Ek olarak; [US-02], [US-03], [US-04], [US-05], [US-06] ve özellikle [US-07] İçerik çakışmaları önlenmeli (niasinamid+C vit., glikolik asit+retinol vb.) kullanıcı hikayeleri planlanmıştır.
- **Gerçekleşen Görevler:** Sprint'in hemen başında veritabanı modellerindeki uyumsuzluktan (AppUser/SkinProfile) kaynaklanan kritik bir hata (bug) nedeniyle 3 günlük bir gecikme yaşanmıştır. Ancak acil mimari toplantısıyla bu kriz hızlıca aşılmış, efor artırılarak planlanan arayüz ve filtreleme görevlerinin tamamı sprint takviminden sapmadan bitirilip "COMPLETED" statüsüne alınmıştır.

Sprint 3

- **Sprint Amacı:** Günlük bakım rutinlerini mobil arayüzde dinamik hale getirmek ve kullanıcı takip/motivasyon (streak) ekranlarını tamamlamak.
- **Planlanan Görevler:** Sabah / akşam rutin ekranı Flutter'da geliştirilmesi ile Takip takvimi ekranı (streak, tamamlanan gün sayısı) görevleri planlanmıştır. İlgili kullanıcı hikayeleri ise; [US-08] Kullanıcı günlük rutinini "tamamlandı" olarak işaretleyebilmeli, [US-09] Kullanıcı takvim üzerinden geçmiş rutin uyumunu görebilmeli ve [US-10] Kullanıcı ürün kataloğunda cilt tipine göre filtreleme yapabilmeli olarak belirlenmiştir.
- **Gerçekleşen Görevler:** Planlanan arayüz tasarımları ve filtreleme özellikleri bitirilerek "COMPLETED" statüsüne alınmıştır.

Sprint 4

- **Sprint Amacı:** Mock (sahte) verilerden canlı (gerçek) API'ye geçiş yapmak, hata (error/loading) durumlarını yönetmek ve uygulamayı teslimata hazır hale getirmek.
- **Planlanan Görevler:** Tüm Flutter ekranlarını gerçek API'ye bağlama (mock datadan gerçeğe geçiş), Hata yönetimi ve loading state'lerin eklenmesi, Flutter Uçtan Uca Test ve Navigasyon İyileştirmeleri ile Sprint retrospektif notlarının yazılması. Çapraz platform denetimi için [US-12] Uygulama iOS ve Android'de sorunsuz çalışabilmeli maddesi eklenmiştir.
- **Gerçekleşen Görevler:** Tüm görevler eksiksiz tamamlanarak (COMPLETED) proje jüri sunumuna hazır hale getirilmiştir.

4.4 Sprint Review Sonuçları

Projenin geliştirme döngüsünde yer alan 4 adet Sprint'in her birinin sonunda, ortaya çıkan "Potansiyel Olarak Teslim Edilebilir Ürün Parçası" (Increment) Product Owner (Ürün Sahibi) ve geliştirme ekibi tarafından incelenmiş, aşağıdaki sonuçlar kayıt altına alınmıştır:

Sprint 1 İncelemesi (Review): Altyapı ve Prototipler

- **Sunulan Çıktı:** ASP.NET Core üzerinde .NET iskeletinin kurulması, veritabanı şemasının belirlenmesi, repo stratejisinin oturtulması ve Flutter proje/navigasyon kurulumu.
- **İnceleme Sonucu:** Veri modeli onaylanmıştır. Product Owner tarafından mobil taraftaki arayüz prototipleri kullanıcı deneyimi (UX) açısından başarılı bulunarak kabul kriterlerini sağlamıştır.

Sprint 2 İncelemesi (Review): Core API ve Kural Motoru

- **Sunulan Çıktı:** Kayıt/onboarding arayüzleri, E2E sistem testleri ve içerik çakışmalarını önleyen (niasinamid + C vitamini vb.) algoritmik kural motoru.
- **İnceleme Sonucu:** Kural motorunun çakışmaları doğru şekilde tespit ettiği görülmüş, hata kodları denetlenmiş ve iş mantığı modülü onaylanmıştır. *Ancak, entegrasyon testlerinin son aşamasında tespit edilen veritabanı kaynaklı hata (HTTP 500), acil çözüm planıyla Sprint 3'ün başlangıç takvimine aktarılmıştır.*

Sprint 3 İncelemesi (Review): Arayüz Entegrasyonu ve Dashboard

- **Sunulan Çıktı:** Takip takvimi (streak) arayüzleri, sabah/akşam rutin ekranları ve katalogda dinamik ürün filtreleme özellikleri.
- **İnceleme Sonucu:** Kullanıcının rutinini "tamamlandı" olarak işaretlemesi test edilmiş, ekran geçişleri onaylanmış ve arayüzün dinamik yetenekleri Product Owner tarafından kabul edilmiştir.

Sprint 4 İncelemesi (Review): UAT ve Nihai Teslimat (Release)

- **Sunulan Çıktı:** Uygulamanın gerçek API verileriyle haberleşmesi, tüm sistemin loading state ve hata (error) yönetimlerinin eklenmesi, iOS ve Android çapraz platform testlerinin başarıyla geçilmesi.
- **İnceleme Sonucu:** Ürünün MVP (Minimum Viable Product) hedeflerine ulaştığı tam onayla teyit edilmiştir. Sistem hiçbir çökme (crash) yaşanmadan jüri sunumuna hazır, kararlı (stable) sürüm olarak ilan edilmiştir.

4.5 Sprint Retrospective Sonuçları

Her sprint sonunda, Scrum Master moderatörlüğünde toplanan ekibimiz "Neyi iyi yaptık?", "Neyi daha iyi yapabildik?" ve "Hangi aksiyonları almalıyız?" soruları etrafında süreç değerlendirmesi yapmıştır. Retrospektif toplantılarından elde edilen ana sonuçlar aşağıda özetlenmiştir:

1. İyi Giden Süreçler ve Güçlü Yönlerimiz:

- **Proje Yönetim Ritüellerine Katılım İstikrarı:** Ekip üyelerinin Daily Scrum, Sprint Planning ve Retrospective toplantılarına eksiksiz katılımı, şeffaf iletişimi ve motivasyonu en üst düzeyde tutmuştur.

- **Sürekli Test (Continuous Testing):** Yalnızca kod yazımına odaklanılmamış; API uç noktalarından mobil arayüz entegrasyonlarına kadar her aşamada birim ve entegrasyon testleri titizlikle yürütülmüştür. Bu sayede hatalar (bug) canlı ortama çıkmadan tespit edilmiştir.
- **Disiplinli Görev Dağılımı:** Product Backlog'da yer alan iş paketleri, ClickUp üzerinden belirlenen efor tahminlemelerine uygun olarak zamanında eritilmiştir.

2. Karşılaşılan Zorluklar ve İyileştirme Alanları:

- **Veritabanı Çakışmaları:** Sprint ortasında AppUser ve SkinProfile varlıklarının (entity) ilişkisel veritabanında uyumsuzluk yaşaması sebebiyle API'nin sürekli HTTP 500 hatası fırlattığı tespit edilmiştir.
- **Sürüm Kontrol Sistemleri (GitHub) Adaptasyonu:** Başlangıç evresinde, branch (dal) yönetimi ve kod birleştirme operasyonlarındaki pratik eksiklikleri, zaman zaman kod çakışmalarına (conflict) ve efor kayıplarına yol açmıştır.
- **Frontend-Backend Entegrasyon İletişimi:** İstemci ve sunucu taraflı geliştiricilerin veri şemaları (JSON) ve API yapısı konusunda kodlama evresinden önce ortak bir sözleşmeye (contract) bağlanmaması, entegrasyon aşamasında son dakika revizyonları gerektirmiştir.
- **DevOps ve Altyapı Krizleri:** Proje sunumuna kısa süre kala bulut (cloud) sunucu hizmetinin iptal olması, canlıya alma (deployment) sürecini tehlikeye atmıştır.

3. Alınan Aksiyonlar ve Süreç İyileştirmeleri:

- **Mimari Revizyon:** Veritabanındaki HTTP 500 krizini çözmek için acil bir mimari toplantı düzenlenmiş; karmaşık ilişkiler temizlenerek sistem tek ve standart bir User modeli üzerine optimize edilmiştir.
- **Altyapı Stratejisi Değişimi:** Sunucu iptali krizine karşı anında DevOps stratejisi değiştirilmiş ve proje, jüri sunumu için tam izole, güvenli bir yerel ağ (Local IP) üzerinden sorunsuz çalışacak şekilde yeniden yapılandırılmıştır.
- **VCS Mentorluğu ve Sözleşme Kültürü:** Yaşanan Github çakışmalarını önlemek adına Scrum Master tarafından ekibe Git/GitHub eğitimleri verilerek merge operasyonları tek elde toplanmıştır. Ayrıca gelecek fazlar (Phase 2) için kodlamaya geçilmeden önce kesinlikle Swagger/API dokümantasyon sözleşmesi (Contract) yapılması kararı alınmıştır.

5. Gerçekleştirim (Implementation)

5.1 Kullanılan Teknolojiler

5.1.1. Frontend (Mobil) Teknolojileri

- Ana Çerçeve (Framework): Flutter
- Programlama Dili: Dart
- Durum Yönetimi (State Management): Riverpod
- Yönlendirme (Routing): GoRouter

- Ağ İstekleri (Networking): HTTP Paketi
- Yerel Veri Saklama: Shared Preferences
- UI ve Tasarım Paketleri: Material Design, Google Fonts, Table Calendar
- Native Entegrasyonlar: Flutter Native Splash, Flutter Launcher Icons
- Backend Bağlantısı: ASP.NET Core Web API (.NET 8) üzerinden REST API

5.1.2. Backend (API) Teknolojileri

- Ana Çerçeve (Framework): ASP.NET Core 8.0 (Web API)
- Programlama Dili: C# (.NET 8)
- Mimari Yaklaşım: Katmanlı Mimari ve MVC Tasarım Deseni (Controllers, Services, Models, Data)
- Veritabanı: SQLite
- ORM: Entity Framework Core
- Güvenlik & Kimlik Doğrulama: JWT (JSON Web Token), BCrypt, Rate Limiting, CORS
- API Dokümantasyonu: Swagger (OpenAPI)
- Veri İşleme (İş Mantığı): Özel Cilt ve Semptom Analiz Servisleri

5.2 Yazılım Bileşenleri

Frontend Mobil Uygulama Modülleri (Flutter)

Proje kapsamında geliştirilen mobil uygulamanın kullanıcı arayüzü katmanı, modüler ve sürdürülebilir bir mimari ile Flutter framework'ü kullanılarak inşa edilmiştir. Geliştirilen temel bileşenler şu şekildedir:

Dashboard (Ana Sayfa) Modülü: Kullanıcının cilt profiline göre dinamik olarak oluşturulan sabah rutini özetini, günlük adım sayısını ve içerik çakışma uyarılarını gösteren ana ekran. IndexedStack widget'ı kullanılarak alt navigasyon sekmeleri arasında geçiş yapılırken her sekmenin durumunun (state) korunması sağlanmıştır.

Profil Yönetim Modülü: Kullanıcının cilt tipi, cilt endişeleri ve hesap bilgilerini görüntüleyen ekran. Çok sayıda cilt endişesi seçildiğinde oluşan taşma (overflow) hatasını önlemek için Wrap widget'ı kullanılarak responsive bir etiket (chip) listesi tasarlanmıştır.

Rutin Takip Modülü: Sabah ve akşam bakım adımlarının Checkbox bileşenleriyle işaretlenebildiği ekran. Kullanıcı bir adımı tamamladığında backend'deki POST /api/tracking/complete uç noktasına asenkron (async/await) istek atılmaktadır. Adımların tamamlanma durumu SharedPreferences ile cihaz üzerinde günlük bazda saklanmakta, her gün otomatik olarak sıfırlanmaktadır.

Takvim ve Streak Görselleştirme Modülü: table_calendar paketi kullanılarak aylık takvim görünümü oluşturulmuş, backend'den GET /api/tracking/{userId}/history ile çekilen veriler işlenerek tamamlanan günler görsel olarak (yeşil işaretleyici) vurgulanmıştır. Ardışık tamamlanan gün sayısını (streak) hesaplayan algoritma, tarih farkları üzerinden geriye dönük olarak çalışmaktadır.

Form Doğrulama Sistemi: Giriş, kayıt ve profil güncelleme ekranlarında TextFormField ve RegExp kullanılarak e-posta formatı, şifre uzunluğu/karmaşıklığı ve boş alan kontrolleri gerçek zamanlı olarak yapılmaktadır. Şifre kriterleri kullanıcı yazarken anlık olarak (AnimatedSwitcher) görsel geri bildirim vermektedir.

HTTP Servis ve API Entegrasyon Katmanı: http paketi kullanılarak AuthService, TrackingService ve ProductService adlı servis sınıfları oluşturulmuştur. Bu katman, backend'e atılan tüm GET/POST isteklerini merkezi olarak yönetmekte; try-catch blokları ile ağ hatalarını yakalayıp kullanıcıya Snackbar bildirimleri göstermektedir.

DermaSmart projesinin sunucu tarafı, kişiselleştirilmiş cilt bakım algoritmalarını işlemek, veri setlerini yönetmek ve istemciye güvenli veri sunmak amacıyla ASP.NET Core Web API mimarisi kullanılarak geliştirilmiştir. Geliştirilen temel backend bileşenleri şu şekildedir:

İçerik Çakışma ve Doğrulama Kural Motoru (Conflict Engine): Projenin en kritik güvenlik katmanı olan bu modül, ConflictService sınıfı altında kurgulanmıştır. Kullanıcıların girdiği ürünlerin aktif bileşenleri (AHA, BHA, Retinol, C Vitamini, Benzoil Peroksit vb.) arasındaki uyumsuzlukları denetler. Farklı dillerde veya ticari isimlerle girilen maddeleri NormalizeIngredient fonksiyonuyla standartlaştırır. Çakışma tespit edildiği anda ürünleri aynı rutine almayarak, birini sabah diğeri akşam rutinine yönlendiren dinamik bir kural motoru olarak çalışır.

```
public bool HasConflict(string firstIngredient, string secondIngredient)
{
    var first = NormalizeIngredient(firstIngredient);
    var second = NormalizeIngredient(secondIngredient);

    if (string.IsNullOrEmpty(first) || string.IsNullOrEmpty(second))
        return false;

    if (first == second)
        return true;

    return IsPair(first, second, "retinol", "vitamin c")
        || IsPair(first, second, "retinol", "aha")
        || IsPair(first, second, "retinol", "bha")
        || IsPair(first, second, "retinol", "benzoyl peroxide")
        || IsPair(first, second, "vitamin c", "aha")
        || IsPair(first, second, "vitamin c", "bha")
        || IsPair(first, second, "vitamin c", "benzoyl peroxide")
        || IsPair(first, second, "aha", "bha")
        || IsPair(first, second, "aha", "benzoyl peroxide")
        || IsPair(first, second, "bha", "benzoyl peroxide");
}
```

Celestia Symptom-İçerik Eşleştirme Modülü (Celestia Service): Kaggle'dan elde edilen 4 farklı ham veri setinin temizlenip birleştirilmesiyle oluşturulan nihai cilt bakım veri setini (CELESTIA_SKINCARE_DATASET_KAGGLE_READY.csv) sunucunun ayağa kalkma (startup) aşamasında RAM üzerine yükleyerek bir sözlük (Dictionary) veri yapısında tutar. Kullanıcının profilindeki cilt endişelerini (sivilce, leke, kuruluk vb.) veri setindeki karşılıklarıyla

eşleştirerek, o cilt problemine bilimsel olarak en iyi gelecek aktif etken maddelerin listesini milisaniyeler düzeyinde tespit eder.

Kişiselleştirilmiş Rutin Oluşturma ve Sıralama Servisi (Morning/Evening Services): Cilt tipine (kuru, yağlı, karma, hassas) ve kullanım zamanına göre ürün seçimi yapan modüldür. MorningRoutineService ve EveningRoutineService sınıflarından oluşur. Sabah rutini için güneşe duyarlılık yaratan asitler ve Retinol elenirken, akşam rutini için de SPF içeren koruyucular elenir. Seçilen ürünler, dermatolojik uygulama sırasına (ince kıvamdan yoğun kıvama) göre GetOrder algoritmasıyla (Temizleyici -> Tonik -> Aktif Serumlar -> Nemlendirici -> Güneş Kremi) sıralanarak listelenir.

Dinamik Rutin Üretimi ve Yönetimi API Katmanı (Routine Controller): /api/routine/generate (POST) ve /api/routine/{userId} (GET) uç noktalarını barındıran katmandır. POST isteği geldiğinde, kullanıcının cilt profili veritabanından sorgulanır, çakışma ve rutin kuralları işletilerek dinamik bir rutin üretilir ve bu rutin ilişkisel RoutineSteps tablosuna kaydedilir. GET isteği ise üretilen bu rutini adım sırasına göre sıralı bir JSON formatında frontend'e (Flutter) servis eder.

```
200
Response body
{
  "userId": 24,
  "skinType": "oily",
  "concerns": [
    "acne",
    "kızarıklık"
  ],
  "morningRoutine": [
    {
      "id": 1,
      "name": "Gentle Cleanser",
      "type": "cleanser",
      "ingredient": "hyaluronik asit",
      "isMorningSuitable": true,
      "isEveningSuitable": true
    },
    {
      "id": 2,
      "name": "Vitamin C Serum",
      "type": "serum",
      "ingredient": "vitamin c",
      "isMorningSuitable": true,
      "isEveningSuitable": true
    },
    {
      "id": 3,
      "name": "Niacinamide Serum",
      "type": "serum",
```

Katalog Arama ve Ürün Filtreleme Modülü (Products Controller): Mobil uygulamadaki "Keşfet/Katalog" bölümünü besleyen API modülüdür. /api/products ucu üzerinden çalışır. İstek parametresi (Query Parameter) olarak cilt tipi (skinType) veya cilt endişesi (concern) aldığında, SQLite veritabanındaki ürün havuzunu anlık olarak filtreleyerek uygun ürünleri ve aktif içeriklerini listeler.

Veri Tabanı Modelleme ve Kaynak Veri Aktarım Katmanı (EF Core & Seeding): Product, SkinProfile ve RoutineStep veri tabanı şemalarının oluşturulduğu ve yönetildiği veri katmanıdır. Entity Framework Core ORM aracı kullanılarak SQL kodu yazılmadan veritabanı işlemleri yürütülür. Uygulama ilk kez çalıştırıldığında, JSON formatındaki ham ürün veri setini (skincare_products_seed.json) okuyarak veritabanındaki Products tablosunu otomatik olarak besleyen (Data Seeding) bir başlangıç mekanizması barındırır.

5.3 Veritabanı Tasarımı

DermaSmart uygulamasının veritabanı altyapısı, hafif, taşınabilir ve hızlı olması sebebiyle SQLite veritabanı motoru üzerine inşa edilmiştir. Veritabanı şeması, Entity Framework Core ORM (Object-Relational Mapper) aracı kullanılarak Code-First yaklaşımla tasarlanmıştır. İlişkisel veri modeli şu temel tabloları ve ilişkileri içermektedir:

Users (Kullanıcılar): Sistemdeki her bir kullanıcının benzersiz kimlik (Id), ad-soyad (FullName), e-posta adresi (Email), şifrelenmiş parola hash'ini (PasswordHash) ve kayıt tarihini (CreatedAt)saklar.

SkinProfiles (Cilt Profilleri): Kullanıcıların anket sonucu elde edilen cilt profillerini tutar. Cilt tipi (SkinType), cilt endişeleri (Concerns) ve yaş aralığı (AgeRange) alanlarını barındırır. Her bir profil, bir kullanıcıya aittir ve Users tablosuyla Bire Bir (One-to-One) ilişkiye sahiptir.

Products (Ürünler): Cilt bakımı için önerilecek ürün kataloğunu barındırır. Ürün adı (Name), kategori (Category), uygun cilt tipleri (SkinTypes), tüm içerikler (Ingredients), fiyat (Price), aktif etken maddeler (ActiveIngredients), kullanım amacı (UsagePurpose), görsel bağlantısı (ImageUrl) ve ürün detay linkini (ProductUrl) içerir.

Ingredients (İçerikler ve Çakışmalar): Kural tabanlı eşleştirme ve çakışma motorunun temelini oluşturur. Etken madde adını (Name), işlevini (Function) ve birlikte kullanılmaması gereken diğer içeriklerin listesini (Conflicts) saklar.

RoutineSteps (Rutin Adımları): Kullanıcıların sabah ve akşam rutin adımlarını tutar. Kullanıcı kimliği (UserId), adım sırası (StepOrder), günün hangi zamanı olduğu (TimeOfDay - morning/evening), ürün kimliği (ProductId) ve oluşturulma tarihini (CreatedAt) barındırır. Users ve products tablolarıyla Bire Çok (One-to-Many) ilişkiye sahiptir.

TrackingLogs (Takip Kayıtları): Kullanıcının günlük rutin adımlarını ne ölçüde tamamladığını izlemek amacıyla tasarlanmıştır. Kullanıcı kimliği (UserId), tamamlanan rutin adım kimliği (RoutineStepId), işlemin yapıldığı tarih (CompletedDate) ve tamamlanma durumunu (IsCompleted) tutar. Users tablosuyla Bire Çok (One-to-Many) ilişkiye sahiptir.

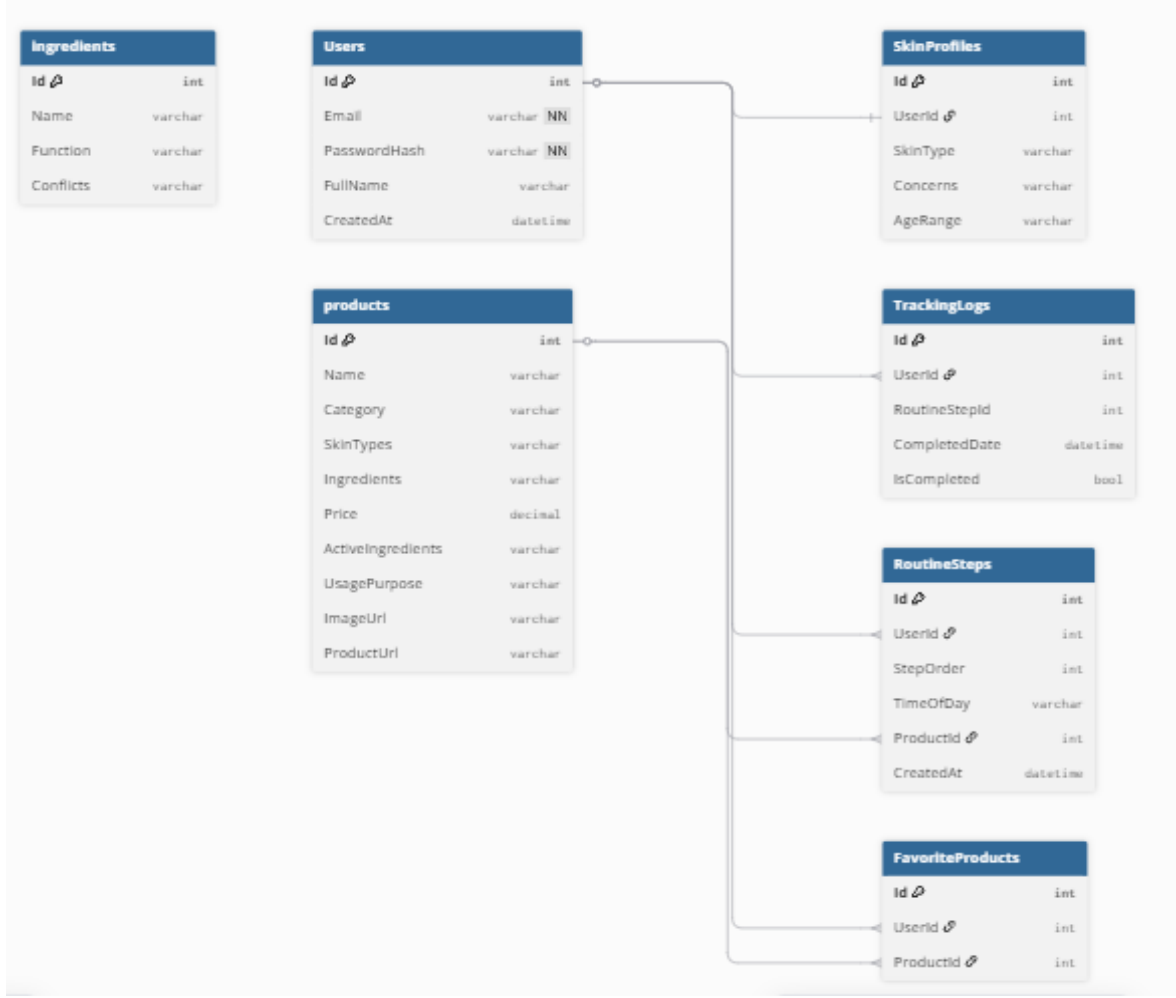
FavoriteProducts (Favori Ürünler): Kullanıcıların beğendikleri ürünleri saklamalarını sağlar. Users ve products tabloları arasında Çoktan Çoğa (Many-to-Many) ilişkiyi çözümleyen bir köprü tablodur.

ER

Diyagramı

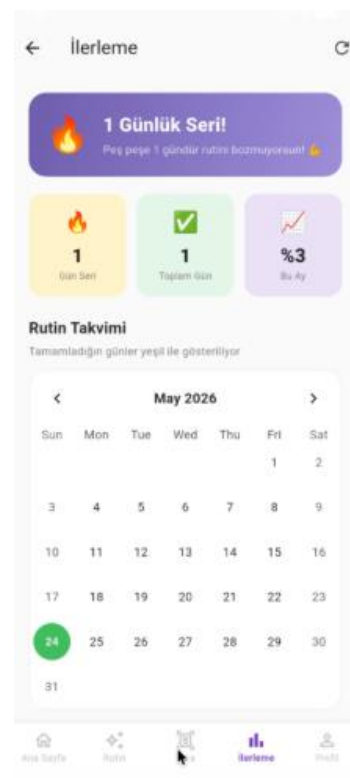
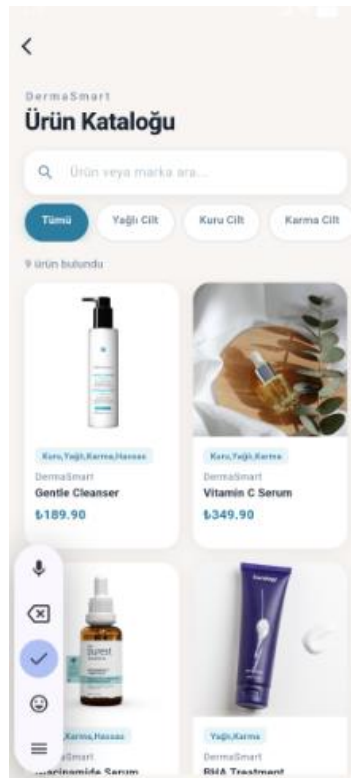
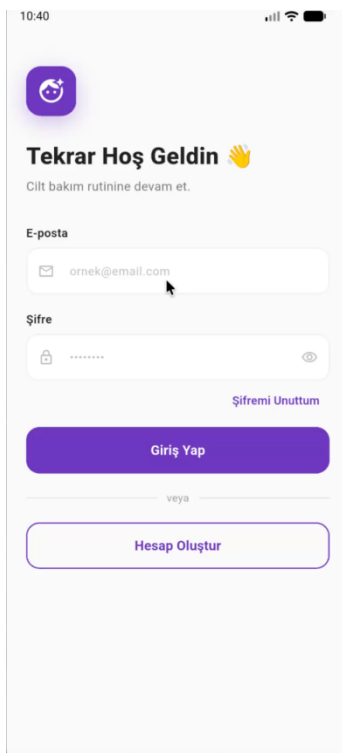
aşağıda

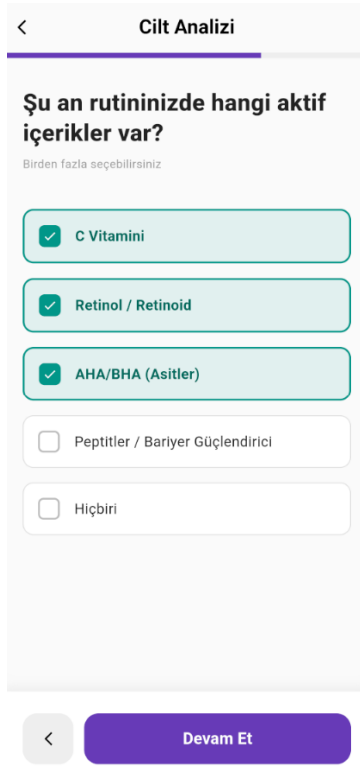
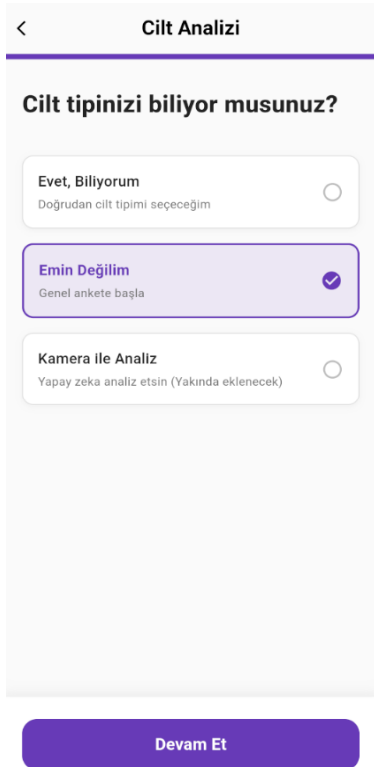
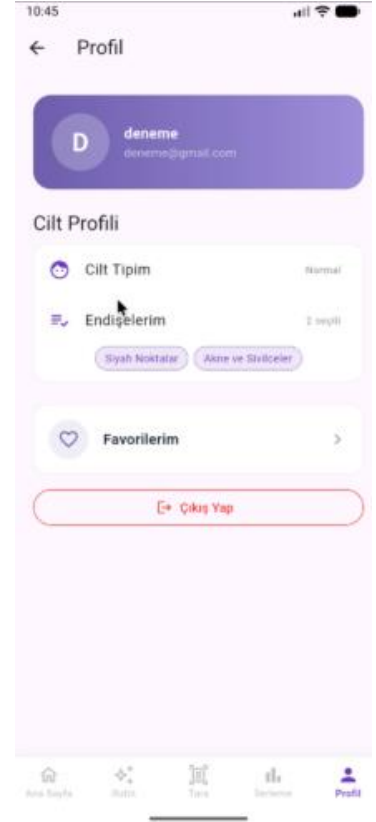
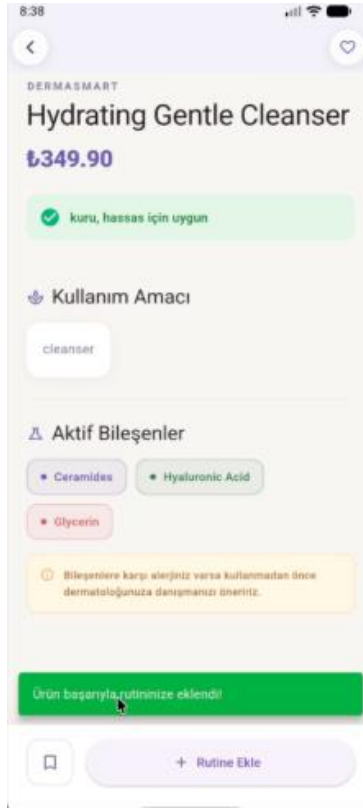
verilmiştir.



5.4 Arayüz Tasarımı

Ekran görüntüleri





6. Doğrulama ve Geçerleme

6.1 Test Planı

Bu proje kapsamında geliştirilen DermaSmart mobil uygulamasının işlevsel doğruluğunu, güvenilirliğini ve kullanıcı gereksinimlerini karşılama düzeyini değerlendirmek amacıyla kapsamlı bir test süreci yürütülmüştür. Test faaliyetleri; birim testleri, entegrasyon testleri, sistem testleri ve kullanıcı kabul testleri olmak üzere dört ana aşamada gerçekleştirilmiştir.

Test sürecinde uygulamanın kullanıcı yönetimi, cilt profili oluşturma, ürün önerileri, rutin oluşturma ve rutin takip modülleri değerlendirilmiştir. Backend servislerinin doğrulanması amacıyla Swagger ve Postman araçları kullanılarak API uç noktalarına çeşitli istekler gönderilmiş, dönen yanıtlar beklenen sonuçlarla karşılaştırılmıştır. Ayrıca veritabanı işlemleri, hata yönetimi mekanizmaları ve kullanıcı senaryoları test edilerek sistemin bütünsel olarak doğru çalıştığı doğrulanmıştır.

Test planının temel hedefleri aşağıdaki şekilde belirlenmiştir:

- Uygulama bileşenlerinin bireysel olarak doğru çalıştığının doğrulanması,
- Modüller arası veri alışverişinin ve API entegrasyonlarının kontrol edilmesi,
- Sistem genelindeki iş akışlarının hatasız şekilde gerçekleştirildiğinin doğrulanması,
- Son kullanıcı gereksinimlerinin karşılandığının teyit edilmesi,
- Olası hata durumlarında sistemin kontrollü ve güvenli şekilde davranmasının sağlanması.

Bu kapsamda gerçekleştirilen testlere ait sonuçlar aşağıdaki bölümlerde detaylı olarak sunulmuştur.

6.2 Birim Testleri

Test Senaryosu	Beklenen Sonuç	Gerçekleşen Sonuç	Durum
Kullanıcı ID doğrulama kontrolü	Geçersiz userId gönderildiğinde hata dönülmesi	API " Geçerli bir userId gönderilmelidir. " mesajı döndürmüştür	Başarılı
Rutin oluşturma algoritmasının çalıştırılması	Kullanıcının cilt profiline uygun rutin oluşturulması	Karma cilt tipi için sabah ve akşam rutinleri başarıyla oluşturulmuştur	Başarılı
Takip kaydı oluşturma işlemi	Tamamlanan adımın veritabanına kaydedilmesi	Tracking kaydı başarıyla oluşturulmuştur	Başarılı
Kullanıcı kayıt ekranından yeni hesap oluşturulması	Kullanıcı bilgilerinin veritabanına kaydedilmesi	API üzerinden kayıt işlemi başarıyla tamamlanmıştır	Başarılı
Aynı adımın aynı tarihte tekrar işaretlenmesi	Yinelenen kayıt oluşturulmaması	API "Step already completed for this date." mesajı döndürmüştür	Başarılı
Geçmiş verisi filtreleme işlemi	Kullanıcıya ait tamamlanan	Doğru kullanıcıya ait geçmiş kayıtları döndürülmüştür	Başarılı

	adımların listelenmesi		
--	------------------------	--	--

6.3 Entegrasyon Testleri

Test Senaryosu	Beklenen Sonuç	Gerçekleşen Sonuç	Durum
Geçersiz e-posta/şifre ile giriş denemesi	Kullanıcıya hata mesajı gösterilmesi, sisteme giriş yapılmaması	API'den 401 hatası alınmış, Snackbar ile kullanıcıya bilgi verilmiştir	Başarılı
Quiz tamamlandıktan sonra cilt profilinin kaydedilmesi	Cilt tipi ve endişelerin backend'e iletilmesi	Profil verisi API'ye başarıyla gönderilmiş ve kaydedilmiştir	Başarılı
Kullanıcı giriş ekranından API'ye giriş isteği gönderilmesi	Kullanıcının doğrulanması ve Dashboard ekranına yönlendirilmesi	API'den başarılı yanıt alınmış ve kullanıcı sisteme giriş yapmıştır	Başarılı
Kullanıcı kayıt ekranından yeni hesap oluşturulması	Kullanıcı bilgilerinin veritabanına kaydedilmesi	API üzerinden kayıt işlemi başarıyla tamamlanmıştır	Başarılı
Ürün kataloğunun API'den yüklenmesi	Ürün listesinin ekranda görüntülenmesi	Ürün verileri başarıyla çekilmiş ve liste halinde gösterilmiştir	Başarılı
Rutin adımının tamamlandı olarak işaretlenmesi	POST /api/tracking/complete isteğinin başarılı olması	İşaretleme işlemi backend'e iletilmiş ve kayıt oluşturulmuştur	Başarılı
Takvim geçmiş verilerinin çekilmesi	GET /api/tracking/{userId}/history verilerinin görüntülenmesi	Tamamlanan günler ve ilerleme bilgileri başarıyla görüntülenmiştir	Başarılı
Sunucu bağlantısının kesilmesi	Uygulamanın çökmeden hata mesajı göstermesi	Hata yakalanmış ve kullanıcıya bilgilendirme mesajı gösterilmiştir	Başarılı

6.4 Sistem Testleri

Test Senaryosu	Beklenen Sonuç	Gerçekleşen Sonuç	Durum
----------------	----------------	-------------------	-------

Backend uygulamasının başlatılması	API servislerinin sorunsuz çalışması	Uygulama localhost:5030 üzerinde başarıyla çalıştırılmıştır	Başarılı
Swagger arayüzünün açılması	Tüm endpointlerin görüntülenebilmesi	Swagger üzerinden endpointler başarıyla görüntülenmiştir	Başarılı
Tracking API uç noktalarının çalıştırılması	Tracking API uç noktalarının çalıştırılması	Tamamlama ve geçmiş işlemlerinin çalışması	Başarılı
Routine Generate API'nin çalıştırılması	Kullanıcıya özel rutin oluşturulması	Kullanıcının cilt profiline uygun rutin başarıyla dönmüştür	Başarılı
Veritabanı migration güncellemesi sonrası sistem kontrolü	Veritabanı hatalarının giderilmesi	Migration uygulanmış ve SQLite kolon hatası çözülmüştür	Başarılı

6.5 Kullanıcı Kabul Testleri

Test Senaryosu	Beklenen Sonuç	Gerçekleşen Sonuç	Durum
Kullanıcının günlük rutinini tamamlaması	Tamamlanan adımın sisteme kaydedilmesi	İşlem başarıyla kaydedilmiş ve geçmişte görüntülenmiştir	Başarılı
Kullanıcının geçmiş rutin kayıtlarını görüntülemesi	Tamamlanan adımların listelenmesi	Geçmiş verileri doğru şekilde görüntülenmiştir	Başarılı
Kullanıcının kişisel cilt rutinini oluşturması	Cilt tipine uygun öneriler sunulması	Sabah ve akşam rutinleri başarıyla oluşturulmuştur	Başarılı
Aynı gün aynı adımın tekrar tamamlanmaya çalışılması	Kullanıcının bilgilendirilmesi	Sistem tekrar kaydı engelleyerek uygun hata mesajı göstermiştir	Başarılı
Kullanıcının sistemle etkileşiminin kesintisiz sürmesi	Hata olmadan işlemlerin tamamlanması	Test edilen tüm API işlemleri başarıyla tamamlanmıştır	Başarılı

7. Proje Yönetimi ve Kalite Güvencesi

Projenin geliştirme yaşam döngüsü boyunca çevik (agile) proje yönetimi pratikleri ile yazılım kalite güvence (QA) standartları eşgüdümlü olarak yürütülmüştür. Kodun sürdürülebilirliği, güvenliği ve takım içi koordinasyon aşağıdaki yöntemlerle sağlanmıştır:

7.1 Git/GitHub Kullanımı

Projenin sürüm kontrol sistemi (VCS) olarak Git, bulut depo olarak ise GitHub kullanılmıştır. Geliştirme sürecinde kod karmaşasını ve çakışmaları (conflict) önlemek adına **Feature Branching (Özellik Dallanması)** stratejisi benimsenmiştir.

- Her bir kullanıcı hikayesi (User Story) ve görev için ana daldan (main) bağımsız yeni dallar (Örn: feature/po-android-test_Kursun4) açılmıştır.
- Ekip üyelerine sürüm kontrol standartları hakkında Scrum Master tarafından mentorluk verilmiş, izole dallarda geliştirilen özellikler tamamlandıktan sonra kontrollü bir şekilde birleştirilmiştir.

Projeye ait tüm açık kaynak kodlar aşağıdaki bağlantılarda şeffaf olarak jüri incelemesine sunulmuştur:

- **Frontend Reposu:** <https://github.com/iamsevval/DermaSmart-FrontEnd>
- **Backend Reposu:** <https://github.com/iamsevval/DermaSmart-BackEnd>

7.2 Kod İnceleme Süreci

Projede kod kalitesini korumak ve hatalı kodların canlı ortama (production) çıkmasını engellemek için sıkı bir kod inceleme süreci işletilmiştir:

- Ana dala (main) doğrudan kod göndermek (commit) engellenmiş, tüm birleştirme işlemleri için **Pull Request (PR)** açılması zorunlu kılınmıştır.
- Açılan PR'lar Scrum Master tarafından mimari standartlara (.NET Core katman kuralları, Flutter widget ağacı optimizasyonları), isimlendirme standartlarına ve iş mantığına uygunluk açısından incelenmiştir.
- Onaylanan (Approve) kodlar ana projeye dahil edilmiş (Merge), testten geçemeyen kodlar ise revizyon talebiyle geliştiriciye geri gönderilmiştir.

7.3 Sürekli Entegrasyon

Geliştirilen birbirinden bağımsız modüllerin (Frontend arayüzleri, Backend API'leri ve Algoritma kural motoru) bir araya geldiğinde sorunsuz çalışmasını garanti altına almak için sürekli entegrasyon prensipleri uygulanmıştır:

- Kodlar birleştirilmeden önce ve sonra, Postman üzerinden API uç noktalarının (Endpoint) 200 OK dönüşleri ve hata fırlatma durumları test edilmiştir.
- Mobil arayüzde mock (sahte) verilerden canlı API verilerine geçiş aşamasında uçtan uca (E2E) entegrasyon testleri yapılmış, HTTP 500 hataları ve UI Freezing (arayüz donması) gibi sorunlar sistem bütününe zarar vermeden izole edilerek çözülmüştür.

7.4 Dokümantasyon Yönetimi

Takım içi asenkron iletişimi güçlendirmek ve projenin teknik hafızasını oluşturmak için çok katmanlı bir dokümantasyon stratejisi izlenmiştir:

- **Görev ve Süreç Dokümantasyonu:** Sprint planlamaları, iş paketleri (Backlog) ve takım atamaları ClickUp çalışma alanı üzerinden dijital olarak kayıt altına alınmıştır.
- **Teknik Dokümantasyon:** Backend API'lerinin parametreleri, dönüş tipleri ve hata kodları (Contract) mobil ekibin rahatça entegre olabilmesi için Swagger (OpenAPI) kullanılarak standartlaştırılmış ve interaktif dokümantasyona dönüştürülmüştür.
- **Test ve Analiz Raporları:** Postman API test raporları, algoritma UAT (Kullanıcı Kabul Testi) sonuçları ve E2E negatif test senaryoları belgelenerek GitHub üzerindeki Docs klasöründe arşivlenmiştir.

8. Etik, Güvenlik ve Sürdürülebilirlik

8.1 Veri Gizliliği

Kullanıcıların cilt analizi verileri, kişisel sağlık ve bakım alışkanlıklarını barındırdığı için yüksek düzeyde gizlilik gerektirmektedir. Projede veri gizliliğini (data privacy) korumamızı sağlayan önlemler şunlardır:

- **Şifre Güvenliği (Password Hashing):** Kullanıcı şifreleri veritabanında asla düz metin (plain text) olarak saklanmamaktadır. BCrypt.Net kütüphanesi kullanılarak tek yönlü kriptografik hash fonksiyonuyla şifrelenmekte ve tuzlanmaktadır (salting). Bu sayede olası veritabanı sızıntılarında bile kullanıcı parolalarının ele geçirilmesi engellenmiştir.
- **Yetkilendirme ve İzolasyon:** Her bir kullanıcının cilt profili ve günlük rutin takip verileri kişiye özeldir. Backend tarafında geliştirilen tüm sorgularda (GET /api/skinprofile/{userId} ve GET /api/tracking/{userId}/history), istek atan kullanıcının JWT token'ındaki userId bilgisi ile talep edilen verinin sahibi olan userId karşılaştırılır. Yetkisiz bir kullanıcının başka bir kullanıcının verilerine erişmesi engellenerek veri izolasyonu sağlanmıştır.

```
[HttpGet("{userId}/history")]
0 references
public async Task<IActionResult> GetHistory(int userId)
{
    var authenticatedUserIdStr = User.FindFirst(System.Security.Claims.ClaimTypes.NameIdentifier)?.Value;
    if (authenticatedUserIdStr == null || int.Parse(authenticatedUserIdStr) != userId)
    {
        return Unauthorized(new { message = "Bu işlemi yapmaya yetkiniz yok." });
    }

    // Kullanıcı var mı kontrolü
    var userExists = await _context.Users.AnyAsync(u => u.Id == userId);
    if (!userExists)
    {
        return NotFound(new { message = "Belirtilen kullanıcı bulunamadı." });
    }
}
```

8.2 Güvenlik Önlemleri

DermaSmart backend mimarisi, yetkisiz erişimlere, veri manipülasyonuna ve aşırı yüke karşı katmanlı bir güvenlik yapısıyla korunmaktadır:

- **JWT (JSON Web Token) Kimlik Doğrulama:** API uç noktalarının güvenliği JWT tabanlı

kimlik doğrulama ile sağlanmıştır. Giriş yapan kullanıcılara benzersiz ve süreli (7 gün geçerli) bir Bearer Token üretilir. Sunucu, gelen isteklerin Authorization başlığındaki token'ın imzasını (SymmetricSecurityKey ve HmacSha256) doğrulayarak yetki kontrolü yapar.

```
Controllers > AuthController.cs > ...
11  space DermaSmart.API.Controllers
15  public class AuthController : ControllerBase
84  public async Task<IActionResult> Login(LoginDto dto)
94  }
95
1 reference
96  private string GenerateJwtToken(User user)
97  {
98      var claims = new[]
99      {
100          new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
101          new Claim(ClaimTypes.Email, user.Email)
102      };
103
104      var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));
105      var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
106
107      var token = new JwtSecurityToken(
108          issuer: _config["Jwt:Issuer"],
109          audience: _config["Jwt:Audience"],
110          claims: claims,
111          expires: DateTime.UtcNow.AddDays(7),
112          signingCredentials: creds
113      );
114
115      return new JwtSecurityTokenHandler().WriteToken(token);
116  }
117 }
```

Request URL

<http://localhost:5030/api/Auth/login>

Server response

Code	Details
200	Response body <pre>{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cy8yMDAxLzA1L2lkZW50aXR5L2NsYWItcy9uYW1laWR1bnRpZm11ci16IjE1IiwiaHR0cDovL3NjaGVtYXMueG1sc29hcC5vcmcvd3MvMjAwNS8wNS9pZGVudG10eS9jbGFpbXMvZW1haWxhZGRyZXNzIjoib3JuZW50aXR5bWVpbi5jb20iLCJleHAiOjE3ODI1NTUwNzcsIm1zcyI6IHR1cm1hcnQilCJhdWQiOiJFZXJtYVNTYXJ0In0.GG0n7Lwd3LDxUih3Dj30qMSEdK7qMZz4SPh3jZ3mLAA", "userId": 15, "fullName": ">x<" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Sat, 20 Jun 2026 10:11:16 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	Success	No links

- **Girdi Doğrulaması (Input Validation):** SQL Injection ve Siteler Arası Betik Çalıştırma (XSS) gibi yaygın web saldırılarını önlemek amacıyla, istemciden gelen tüm veriler DTO (Data Transfer Object) katmanında validasyon nitelikleriyle (Attributes) denetlenir (örn: [Required], [EmailAddress], [MaxLength]). Geçersiz formatta veya aşırı uzun veri girişleri daha API kapısında standart hata mesajlarıyla reddedilir.

```

Controllers > SkinProfileController.cs > ...
9 namespace DermaSmart.API.Controllers
14 public class SkinProfileController : ControllerBase
25 public async Task<IActionResult> CreateSkinProfile([FromBody] SkinProfileDto dto)
41 // CONCERNS REQUIRED TEST
42 if (dto.Concerns == null || dto.Concerns.Count == 0 || dto.Concerns.All(string.IsNullOrEmpty))
43     return BadRequest(new { message = "En az bir cilt endişesi (concern) girmek zorunludur." });
44
45 // LONG INPUT TEST
46 if (dto.Concerns.Any(c => c.Length > 100))
47     return BadRequest(new { message = "Concern değeri çok uzun." });
48
49 // VALID SKIN TYPE TEST
50 var validSkinTypes = new List<string> {
51     "dry", "oily", "combination", "normal", "sensitive",
52     "kuru", "yağlı", "yaglı", "karma", "hassas"
53 };
54 if (!validSkinTypes.Contains(dto.SkinType!.Trim().ToLower()))
55     return BadRequest(new { message = "Geçersiz skin type." });
56
57 // CONFLICT LOGIC TEST
58 var skinTypeParts = dto.SkinType.ToLower().Split(',').Select(s => s.Trim()).ToList();
59 if ((skinTypeParts.Contains("oily") || skinTypeParts.Contains("yağlı")) && (skinTypeParts.Contains("dry")
60     return BadRequest(new { message = "Birbiriyle çakışan cilt tipleri gönderilemez." });
61
62 // VALID AGE RANGE TEST
63 var validAgeRanges = new List<string> {
64     "18-24", "18 - 24",
65     "25-34", "25 - 34",
66     "35-44", "35 - 44",
67     "45 ve üzeri"
68 };
69

```

• **Hız Sınırlandırma (API Rate Limiting):** Hizmet dışı bırakma (DDoS) saldırılarını ve kaba kuvvet (brute-force) giriş denemelerini engellemek amacıyla .NET Core Rate Limiting kütüphanesi kullanılarak Fixed Window Rate Limiter kurulmuştur. Kullanıcı IP adresi veya ID'si bazında saniyede en fazla 10 isteğe izin verilmekte, sınır aşımında özelleştirilmiş TOO_MANY_REQUESTS JSON hata paketi ile 429 durum kodu dönülmektedir.

```

Program.cs
59 builder.Services.AddRateLimiter(options =>
61     options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
62     options.OnRejected = async (context, token) =>
63     {
64         context.HttpContext.Response.StatusCode = StatusCodes.Status429TooManyRequests;
65         context.HttpContext.Response.ContentType = "application/json";
66         await context.HttpContext.Response.WriteAsync(JsonSerializer.Serialize(new
67         {
68             success = false,
69             errorCode = "TOO_MANY_REQUESTS",
70             message = "Çok fazla istek gönderdiniz. Lütfen saniyede en fazla 10 istek göndermeyi deneyin.",
71             statusCode = 429
72         })), cancellationToken: token);
73     });
74
75 options.GlobalLimiter = PartitionedRateLimiter.Create<HttpContext, string>(httpContext =>
76 {
77     var partitionKey = httpContext.User.FindFirst(System.Security.Claims.ClaimTypes.NameIdentifier)?.Value
78     ?? httpContext.Connection.RemoteIpAddress?.ToString()
79     ?? "global";
80
81     return RateLimitPartition.GetFixedWindowLimiter(
82         partitionKey: partitionKey,
83         factory: partition => new FixedWindowRateLimiterOptions
84         {
85             AutoReplenishment = true,
86             PermitLimit = 10,
87             QueueLimit = 0,
88             Window = TimeSpan.FromSeconds(1)
89         });
90 });
91

```

• **CORS ve Hata Yönetimi (Exception Handling):** Tarayıcı tabanlı yetkisiz kaynak erişimlerini engellemek için CORS kuralları yapılandırılmıştır. Ayrıca olası sistem çökmelerinde sunucunun iç detaylarının ve hassas bilgilerin sızmasını önlemek amacıyla global bir

UseExceptionHandler middleware'i yazılmış, 500 Internal Server Error hataları sadeleştirilerek maskelenmiştir.

8.3 Etik Değerlendirmeler

DermaSmart bir cilt analizi ve rutin öneri uygulamasıdır. Projenin etik standartlara uygunluğu şu hususlar gözetilerek sağlanmıştır:

- **Tıbbi Teşhis ve Tavsiye Sınırı (Kapsam Sınırı):** Uygulama kesinlikle tıbbi bir teşhis (dermatolojik tedavi) vaat etmemekte, sadece kullanıcıların beyan ettiği cilt tipine uygun kozmetik bakım adımları önermektedir. Etik sorumluluk gereği kullanıcılara bunun tıbbi bir muayene olmadığı, ciddi cilt rahatsızlıklarında uzman bir dermatoloğa başvurulması gerektiği Swagger dokümantasyonlarında ve sistem vizyonunda açıkça belirtilmiştir.
- **Kullanıcı Sağlığını Koruma ve Çakışma Motoru:** Kullanıcının cildine zarar verebilecek, birbirini bloke eden veya tahriş riskini artıran (örn. C Vitamini ile Glikolik Asit çakışması) aktif bileşenlerin sabah ve akşam rutinlerinde aynı anda önerilmesi kural tabanlı çakışma motorumuzla engellenmiştir. Bu durum, kullanıcının cilt bariyerinin zarar görmesini önleyen etik bir tasarım tercihidir.

8.4 Toplumsal ve Çevresel Etkiler

- **Toplumsal Fayda:** Bireylerin kendi cilt tiplerini doğru tanımalarını sağlayarak bilinçsiz ve kulaktan dolma kozmetik ürün kullanımlarını azaltır. Cilt sağlığı bilincini artırarak toplumsal öz bakım kalitesine katkıda bulunur.
- **Çevresel Sürdürülebilirlik:** Yanlış ürün satın alımından kaynaklanan kozmetik tüketim israfını (ve dolayısıyla plastik/cam ambalaj atıklarını) azaltarak çevresel sürdürülebilirliğe dolaylı olarak katkı sağlar. Ayrıca backend tarafında SQLite ve hafif .NET Core mimarisi tercih edilerek, bulut sunucu kaynak tüketimi ve dolayısıyla veri merkezlerinin karbon ayak izi minimize edilmiştir.

9. Sonuçlar ve Değerlendirme

9.1 Hedeflerin Gerçekleşme Durumu

DermaSmart projesinin MVP (Minimum Viable Product) aşaması için belirlenen tüm temel hedeflere ulaşılmıştır. Kullanıcıların cilt profillerini analiz eden kural tabanlı uzman sistem (içerik çakışma motoru) başarıyla çalıştırılmış, ASP.NET Core tabanlı güvenli bir REST API inşa edilmiş ve Flutter kullanılarak geliştirilen mobil arayüz ile uçtan uca entegrasyon sağlanmıştır. Proje, çapraz platform desteğiyle hem iOS hem de Android ortamlarında sorunsuz çalışacak şekilde jüri sunumuna hazır hale getirilmiştir.

9.2 Başarı Ölçütleri

Projenin başarısını kanıtlayan temel ölçütler şunlardır:

- **Sürekli Test (Continuous Testing) ve Kalite Güvencesi:** Geliştirme fazları boyunca API uç noktalarından mobil arayüz entegrasyonlarına kadar yürütülen birim ve E2E testleri sayesinde veri tabanı uyumsuzlukları canlıya çıkmadan engellenmiştir.

- **Agile ve Scrum Uyumu:** Ekip üyelerinin Daily Scrum, Sprint Planning ve Retrospective toplantılarına yüksek katılımı, şeffaf iletişimi ve süreç sahiplenme bilincini en üst düzeyde tutmuştur.
- **Disiplinli Görev Teslimi:** Sprint havuzuna (Backlog) alınan iş paketlerinin, belirlenen efor tahminlemelerine uygun olarak ve projenin takviminden sapmadan tamamlanması.

9.3 Karşılaşılan Problemler

- **Veritabanı Çakışması:** Geliştirme ortasında AppUser ve SkinProfile varlıklarının (entity) ilişkisel uyumsuzluğu nedeniyle API'nin sürekli HTTP 500 hatası fırlatması.
- **DevOps ve Altyapı Krizleri:** Sunum öncesi bulut (cloud) sunucu hizmetinin iptal edilmesi ve sistem altyapısı gibi DevOps sorumluluklarının sıralandırılmasında yaşanan zorluklar.
- **Sürüm Kontrol Sistemleri Adaptasyonu:** Başlangıç evresinde Git/GitHub kullanımındaki (özellikle branch yönetimi ve merge operasyonları) pratik eksikliklerin kod çakışmalarına (conflict) ve efor kayıplarına yol açması.
- **Frontend-Backend İletişimi:** Veri şemaları ve API yapıları konusunda kodlama öncesi ortak bir sözleşmeye (contract) bağlanılmaması sebebiyle entegrasyon aşamasında son dakika revizyonları gerekmesi.

9.4 Öğrenilen Dersler

- **Teknik Şema Sözleşmeleri (API Contract):** Gelecek projelerde kodlamaya geçilmeden önce Frontend ve Backend ekipleri arasında JSON yanıt şemalarını, API uç noktalarını ve hata kodlarını standartlaştıran bağlayıcı bir dokümantasyon oluşturulmasının mutlak bir zorunluluk olduğu anlaşılmıştır.
- **VCS Oryantasyonu:** Yeni projelere başlanmadan önce tüm ekibe Git/GitHub iş akışları ve takım içi kodlama standartları üzerine teknik adaptasyon süreci yürütülmesinin önemi kavranmıştır.
- **DevOps Bağımlılık Yönetimi:** İş paketleri dağıtılırken diğer birimleri bloke etme riski taşıyan altyapı işlerine ve görevler arası bağımlılıklara (dependency) öncelik verilmesi gerektiği tecrübe edilmiştir.
- **Kriz Yönetimi ve Çeviklik:** Veritabanı modelinin acil bir toplantıyla tek bir standart User modeline indirgenmesi ve iptal olan sunucu yerine izole, güvenli yerel ağ (Local IP) stratejisine geçilmesi, ekibin çevik (agile) problem çözme yeteneğini ve operasyonel mükemmelliğini kanıtlamıştır.

10. Gelecek Çalışmalar

DermaSmart uygulamasının mevcut MVP sürümü, Faz 1 hedeflerine ulaşmış, çalışan bir prototip olarak tamamlanmıştır. Projenin bir sonraki geliştirme döngüsünde planlanan geliştirmeler şu şekildedir:

AI Cilt Analizi Arayüzünün Tamamlanması: Şu an yer tutucu olarak bulunan "Cilt Tarama" ekranının, geliştirilmekte olan kamera tabanlı görüntü sınıflandırma modeli backend'de hazır olduğunda gerçek zamanlı analiz arayüzüne (kamera önizleme, analiz sonucu kartları, akne/kızarıklık/nem skorları) kavuşturulması hedeflenmektedir.

Bulut Altyapısına Geçiş Desteği: Uygulamanın şu anda yerel ağ (Local IP) üzerinden çalışan baseUrl yapılandırmasının, Azure/AWS bulut altyapısına geçiş sürecinde ortam değişkenleri üzerinden yönetilecek şekilde yeniden düzenlenmesi planlanmaktadır.

Push Bildirim Arayüzü: Firebase Cloud Messaging (FCM) ve APNs entegrasyonu ile kullanıcılara sabah/akşam rutin hatırlatmaları gönderilmesi hedeflenmektedir. Frontend tarafında bildirim izinleri, bildirim geçmişi ve hatırlatma saat tercihlerinin yönetildiği bir ayarlar ekranının geliştirilmesi planlanmaktadır.

Alışveriş Modülü Arayüzü: Mevcut ürün kataloğu ekranındaki "Rutine Ekle" özelliğinin, sepet yönetimi ve iyzico/Stripe güvenli ödeme altyapısıyla bütünleşik bir alışveriş akışına (sepet, ödeme, sipariş takip ekranları) dönüştürülmesi planlanmaktadır.

Dermatolog Danışma Modülü Arayüzü: Kullanıcıların cilt analiz sonuçlarını uzman dermatologlarla paylaşabildiği, randevu/mesajlaşma tabanlı bir danışma ekranının frontend tarafında tasarlanması hedeflenmektedir.

11. Program Çıktıları Değerlendirmesi

Program Çıktısı	Kanıt
Problem Analizi	Gereksinim Analizi
Tasarım Yeteneği	UML ve Mimari Tasarım
Modern Araç Kullanımı	GitHub, ClickUp, Postman, Visual Studio, VS Code, Swagger
Takım Çalışması	Scrum Süreci
İletişim	Toplantılar ve Sunumlar
Etik ve Mesleki Sorumluluk	Bölüm 8
Proje Yönetimi	Sprint Planları ve ClickUp Takibi

Kaynakça

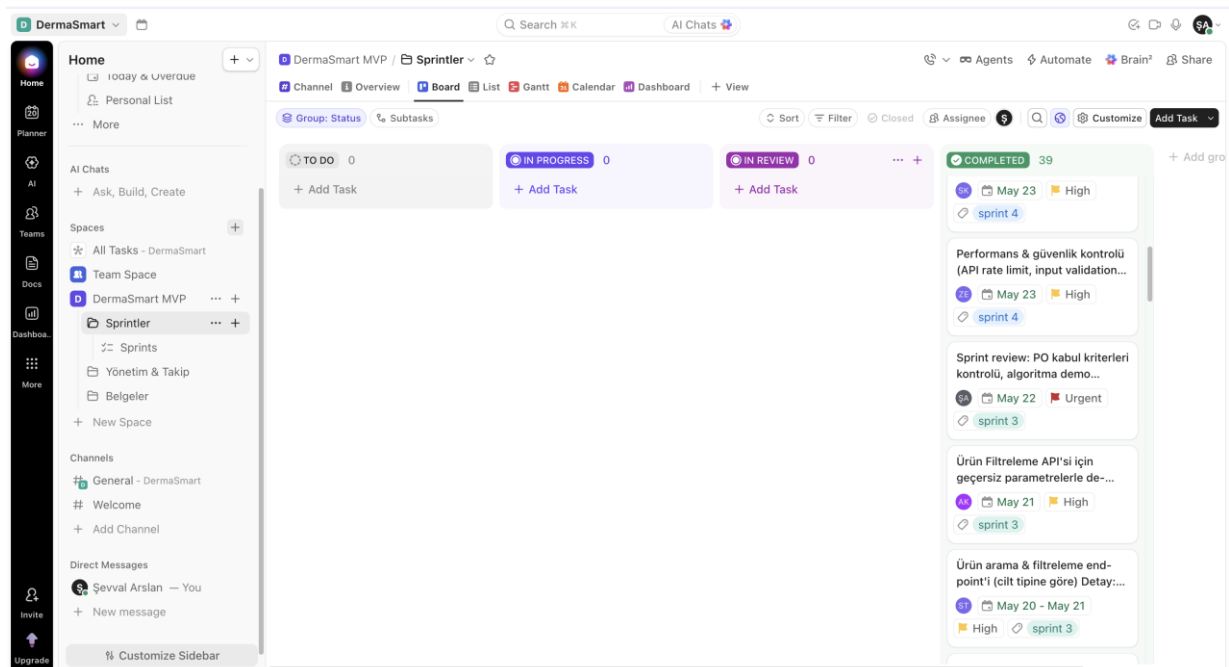
[1] K. Schwaber ve J. Sutherland, "The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game," Scrum.org, 2020. [Çevrimiçi]. Erişim: <https://scrumguides.org/scrum-guide.html>.

- [2] Google, "Flutter Architectural Overview," Flutter Documentation, 2026. [Çevrimiçi]. Erişim: <https://docs.flutter.dev/resources/architectural-overview>.
- [3] Microsoft, "ASP.NET Core Fundamentals," Microsoft Learn, 2026. [Çevrimiçi]. Erişim: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/>.
- [4] Microsoft, "Entity Framework Core: Object-Relational Mapper for .NET," Microsoft Learn, 2026. [Çevrimiçi]. Erişim: <https://learn.microsoft.com/en-us/ef/core/>.
- [5] SQLite Consortium, "SQLite: A Self-Contained, Serverless, Zero-Configuration, Transactional SQL Database Engine," SQLite.org, 2026. [Çevrimiçi]. Erişim: <https://www.sqlite.org/about.html>.
- [6] M. Jones, J. Bradley, ve N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), Mayıs 2015. [Çevrimiçi]. Erişim: <https://datatracker.ietf.org/doc/html/rfc7519>.
- [7] BcryptNet, "BCrypt.Net - Bringing updates to the original bcrypt package," GitHub, 2026. [Çevrimiçi]. Erişim: <https://github.com/BcryptNet/bcrypt.net>
- [8] E. Vadakkan, "Celestia Dermatology Skincare Treatment Dataset," Kaggle, 2026. [Çevrimiçi]. Erişim: <https://www.kaggle.com/datasets/ebin vadakkan/celestia-dermatology-skincare-treatment-dataset> (Proje gereksinimleri doğrultusunda veri ön işleme aşamasında [9], [10] ve [11] numaralı ham veri setleriyle birleştirilmiş ve normalize edilerek kullanılmıştır).
- [9] Amaboh, "Skin Care Product Ingredients (INCI List)," Kaggle, 2026. [Çevrimiçi]. Erişim: <https://www.kaggle.com/datasets/amaboh/skin-care-product-ingredients-inci-list>
- [10] Eward96, "Skincare Products and their Ingredients," Kaggle, 2026. [Çevrimiçi]. Erişim: <https://www.kaggle.com/datasets/eward96/skincare-products-and-their-ingredients>
- [11] Kingabzpro, "Cosmetics Datasets," Kaggle, 2026. [Çevrimiçi]. Erişim: <https://www.kaggle.com/datasets/kingabzpro/cosmetics-datasets>
- [12] S. Seit , C. Bredoux, D. Compan, H. Zucchi, D. Lombard, C. Medaisko, ve A. Fourtanier, "Histological evaluation of a topically applied retinol-vitamin C combination," Skin Pharmacology and Physiology, cilt 18, sayı 2, ss. 81-87, Mart-Nisan 2005, doi: 10.1159/000083708.
- [13] R. Narsa vd., "A Comprehensive Review of the Strategies to Reduce Retinoid-Induced Skin Irritation in Topical Formulation," Dermatology Research and Practice, cilt 2024, Madde No. 5551774, Ağustos 2024, doi: 10.1155/2024/5551774.
- [14] J. Hartman vd., "A pilot study examining a double-conjugated, retinoid-based skincare regimen for darker, blemish-prone skin," Journal of Cosmetic Dermatology, cilt 23, sayı 6, 2024, doi: 10.1111/jocd.16141.

Ek-1 Sprint Backlogları

COMPLETED 12						
Name	Pri...	AI	Assignee	Status		
✓ [US-01] Kullanıcı sisteme kayıt olabilmeli ve giriş yapabilmeli sprint 1 sprint 2	Urgent		ZE +2	COMP...		
✓ [US-02] Kullanıcı cilt tipi anketini doldurabilmeli (kuru/yağlı/karma/hassas) sprint 2 sprint 3	High		ZE +2	COMP...		
✓ [US-03] Kullanıcı cilt problemlerini çoklu seçebilmeli (akne, leke, kuruluk, hassasiyet vb.) sprint 2 sprint 3	High		ZE +2	COMP...		
✓ [US-04] Sistem cilt profiline göre sabah rutini oluşturabilmeli sprint 2 sprint 3	High		ST SK	COMP...		
✓ [US-05] Sistem cilt profiline göre akşam rutini oluşturabilmeli sprint 2 sprint 3	High		ST SK	COMP...		
✓ [US-06] Rutin adımları doğru uygulama sırasıyla listelenmeli sprint 2	High		ST	COMP...		
✓ [US-07] İçerik çıkışmaları önlenmeli (niasinamid+C vit., glikolik asit-retinol vb.) sprint 2	Urgent		ST AK	COMP...		
✓ [US-08] Kullanıcı günlük rutini "tamamlandı" olarak işaretleyebilmeli sprint 3	Normal		AK +2	COMP...		
✓ [US-09] Kullanıcı takvim üzerinden geçmiş rutin uyumunu görebilmeli sprint 3	Normal		SK ZE	COMP...		
✓ [US-10] Kullanıcı ürün kataloğunda cilt tipine göre filtreleme yapabilmeli sprint 3	Low		ZE SK	COMP...		
✓ [US-11] Sistem kullanıcı verilerini şifreli ve güvenli saklayabilmeli sprint 1 sprint 2	Urgent		ZE AK	COMP...		
✓ [US-12] Uygulama iOS ve Android'de sorunsuz çalışabilmeli sprint 4	Normal		SK AK	COMP...		

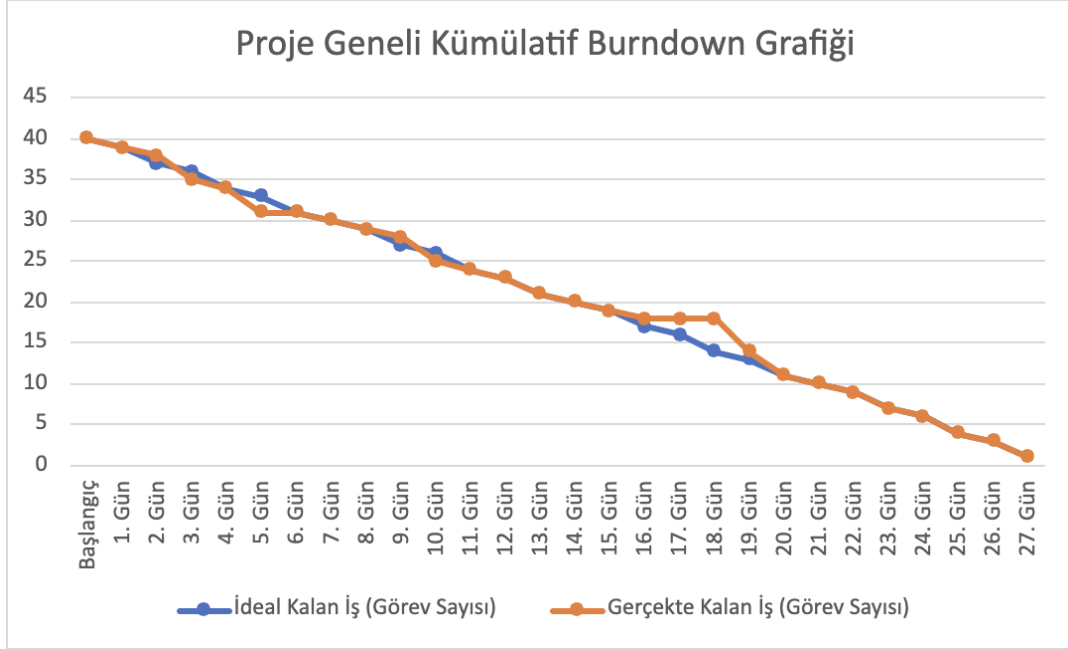
Görsel 1.1- ClickUp üzerinden yönetilen Sprint planlamalarına ait iş paketleri ve atamalar



Görsel 1.2 - ClickUp Sprint Yönetim Panosu (Sprintler Board): Proje genelinde Sprint 1-4 boyunca planlanan ve tamamlanan görevlerin ClickUp üzerinden Board görünümünde takip edilmesi. "To Do", "In Progress", "In Review" ve "Completed" durum sütunlarıyla iş paketlerinin akışı; her görevin sorumlu kişi (assignee), öncelik seviyesi (High/Urgent) ve ilgili sprint etiketiyle birlikte düzenli olarak yönetildiği görülmektedir.

Bağlantı adresi: [ClickUp Public Board](#)

Ek-2 Burndown Grafikleri



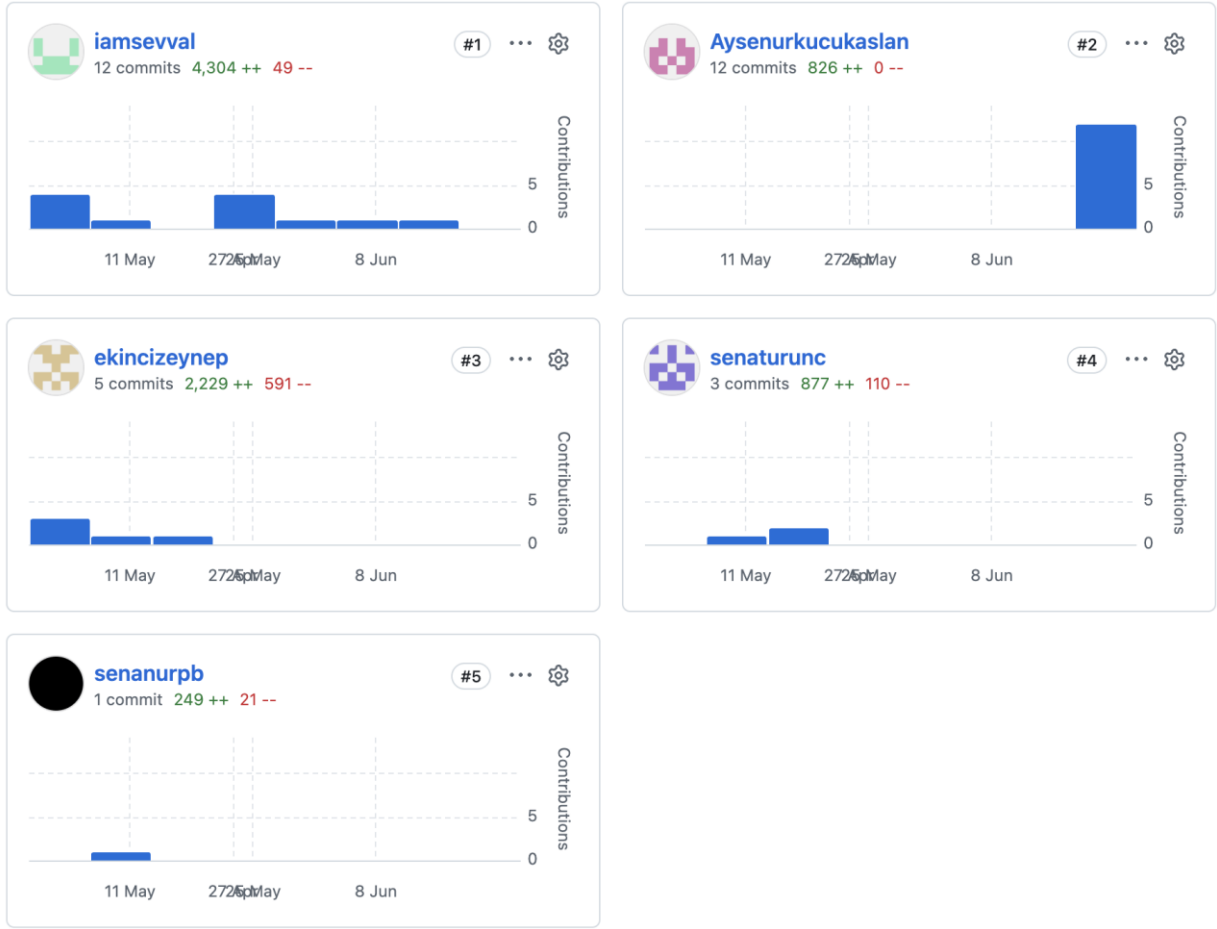
Görsel 2.1 - Proje Geneli Kümülatif Burndown Grafiği

Ek-3 GitHub İstatistikleri

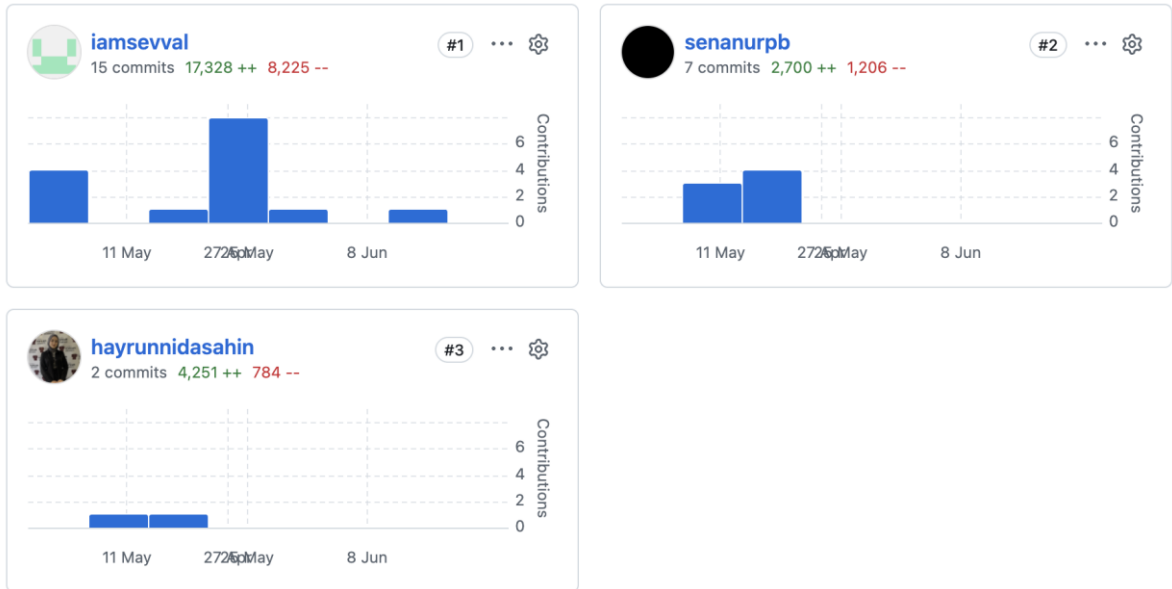
Bu bölümde projenin versiyon kontrol sistemi (GitHub) üzerindeki geliştirme istatistikleri, kod inceleme operasyonları ve ağ (network) dallanma stratejileri hem Frontend hem de Backend depoları (repository) için kanıtlarıyla sunulmuştur.

1. Geliştirici Katkı (Contributors) Grafikleri

Bu görseller, takım üyelerinin proje havuzuna yaptıkları kod ekleme/çıkarma (addition/deletion) işlemlerini ve efor yoğunluklarını zamana bağlı olarak göstermektedir.



Görsel 3.1 - DermaSmart Backend (ASP.NET Core) Reposu: Geliştirici ekibin efor dağılımını kod ekleme/silme metriklerini ve zamana bağlı commit istatistiklerini gösteren "Contributors" grafiği.



Görsel 3.2 - DermaSmart Frontend (Flutter) Reposu: Kullanıcı arayüzü ve API entegrasyonu süreçlerinde takım üyelerinin eşzamanlı kodlama aktivitelerini gösteren "Contributors" istatistikleri.

2. Kod İnceleme ve Birleştirme (Pull Requests - Merged)

Bu görseller, "Feature Branching" (Özellik Dallanması) stratejisinin nasıl işletildiğini ve Scrum Master onayıyla (Code Review) yapılan güvenli kod entegrasyonlarını (Merge) kanıtlamaktadır.

☐	☐ 0 Open ☒ 12 Closed	Author ▾	Label ▾	Projects ▾	Milestones ▾	Reviews ▾	Assignee ▾	Sort ▾
☐	Test/final qa reports ayse5 #12 by iamseval Owner was merged 11 minutes ago • Review required							
☐	Release/test sprint4 stable api kursun3 #11 by iamseval Owner was merged last month • Review required							
☐	feat: add rate limiting, input validation, CORS config and security e... #10 by iamseval Owner was merged last month • Review required							
☐	Fix symptom matching validation #9 by iamseval Owner was merged on May 21 • Review required							
☐	feat: add product search API and seed data #8 by iamseval Owner was merged on May 21 • Review required							
☐	Add routine API and migration updates #7 by iamseval Owner was merged on May 17 • Review required							
☐	feat: add routine tracking API and logs migration #6 by iamseval Owner was merged on May 17 • Review required							
☐	Update backend APIs and improve user profile management #5 by iamseval Owner was merged on May 14 • Review required							
☐	fix: resolve backend logic errors and update controllers #4 by iamseval Owner was merged on May 12 • Review required							
☐	feat: Integrate Skincare Routine Algorithms (Celestia Dataset) #3 by iamseval Owner was merged on May 9 • Review required							
☐	feat: Add Skin Profile Form API Endpoints (POST/GET) #2 by iamseval Owner was merged on May 8 • Review required							
☐	Auth API Refactoring and Bug Fixes #1 by iamseval Owner was merged on May 8 • Review required							

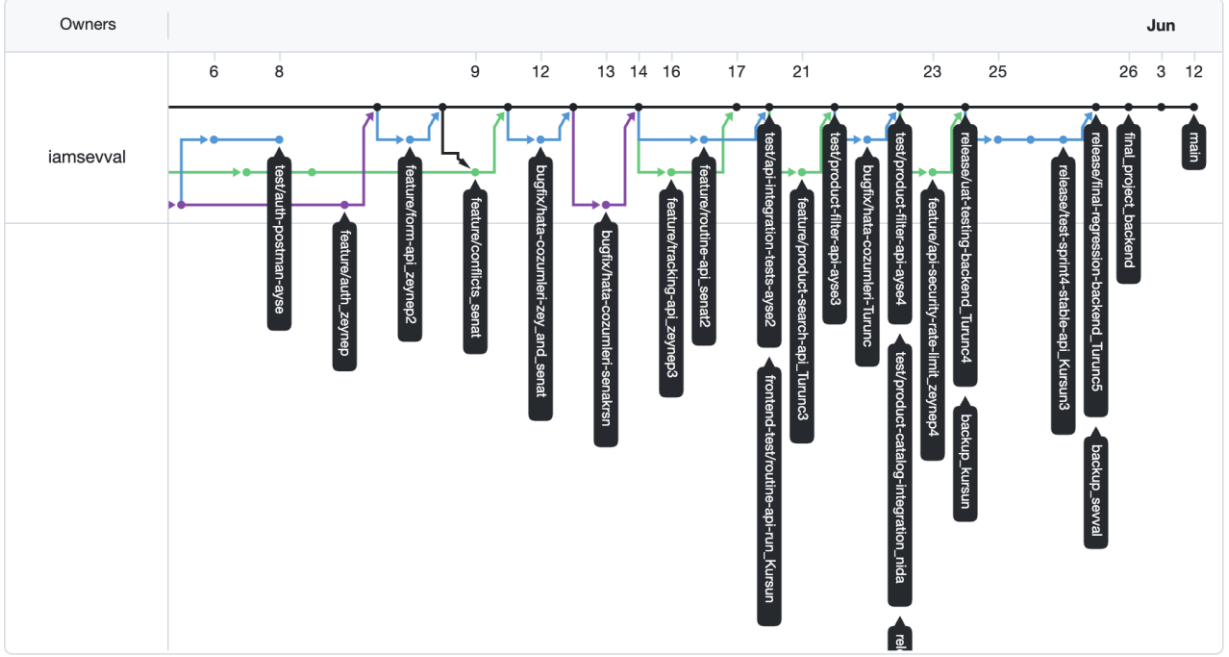
Görsel 3.3 - Backend Reposu Pull Request (PR) Geçmişi: Feature (Özellik) dallarından ana dala (main) yapılan isteklerin incelenerek (Code Review) güvenli bir şekilde birleştirilmesi (Merged) operasyonları.

☐	☐ 0 Open ☒ 7 Closed	Author ▾	Label ▾	Projects ▾	Milestones ▾	Reviews ▾	Assignee ▾	Sort ▾
☐	Feature/po android test kursun5 #7 by iamseval Owner was merged 3 weeks ago							
☐	Feature/po android test kursun4 #6 by iamseval Owner was merged last month							
☐	Feature/product catalog nida #5 by iamseval Owner was merged last month							
☐	feat: complete routine and calendar tracking, fix capitalize bug and ... #4 by iamseval Owner was merged last month							
☐	Feature/routine UI senak2 #3 by iamseval Owner was merged last month							
☐	Bugfix/hata cozumleri senakrsn #2 by iamseval Owner was merged on May 14							
☐	feat: Integrate Auth and Skin Profile APIs to Flutter UI #1 by iamseval Owner was merged on May 9							

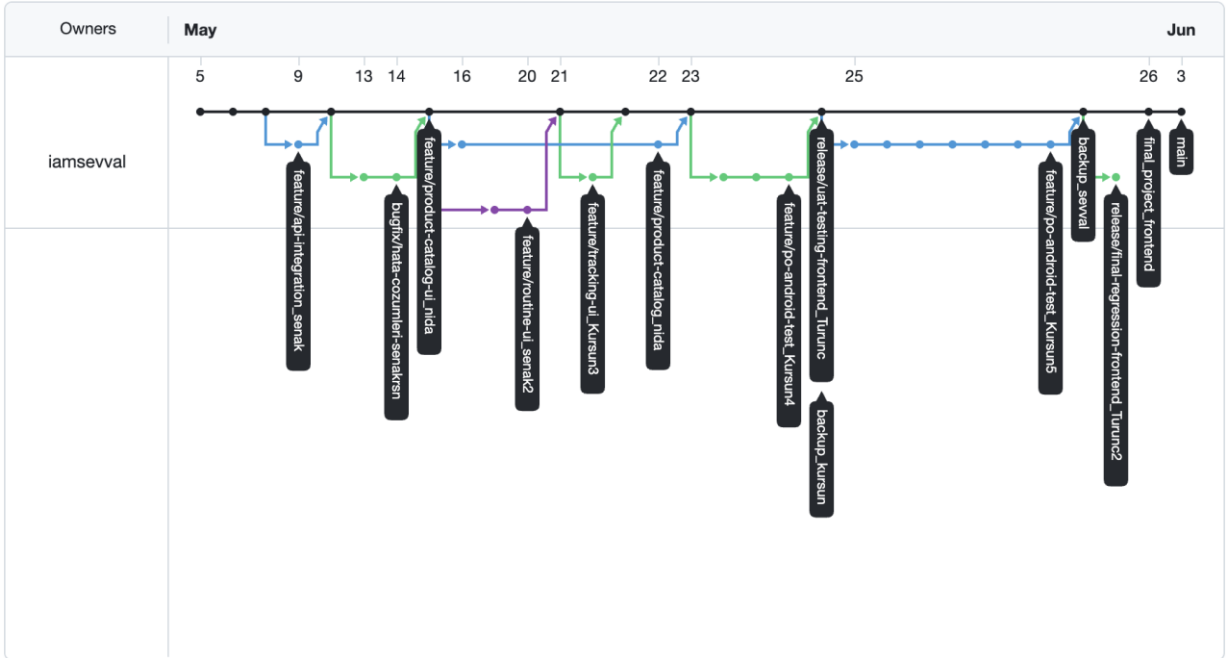
Görsel 3.4 - Frontend Reposu Pull Request (PR) Geçmişi: Geliştiricilerin izole dallarda (branch) tamamladığı arayüz görevlerinin, kod çakışması (conflict) olmadan ana projeye entegrasyon kayıtları.

3. Sürüm Kontrol Ağacı (Network Graph)

Bu görseller, projenin eşzamanlı geliştirme döngüsünde kullanılan dallanma hiyerarşisini (branching strategy) ve düğüm noktalarını görselleştirmektedir.



Görsel 3.5 - Backend Sürüm Kontrol Ağacı (Network Graph): Proje mimarisi ve API uç noktaları geliştirilirken açılan bağımsız özellik dallarının (feature branches) ve ana hat (main) üzerindeki yapılandırılmış birleşme (merge) düğümlerinin görünümü.



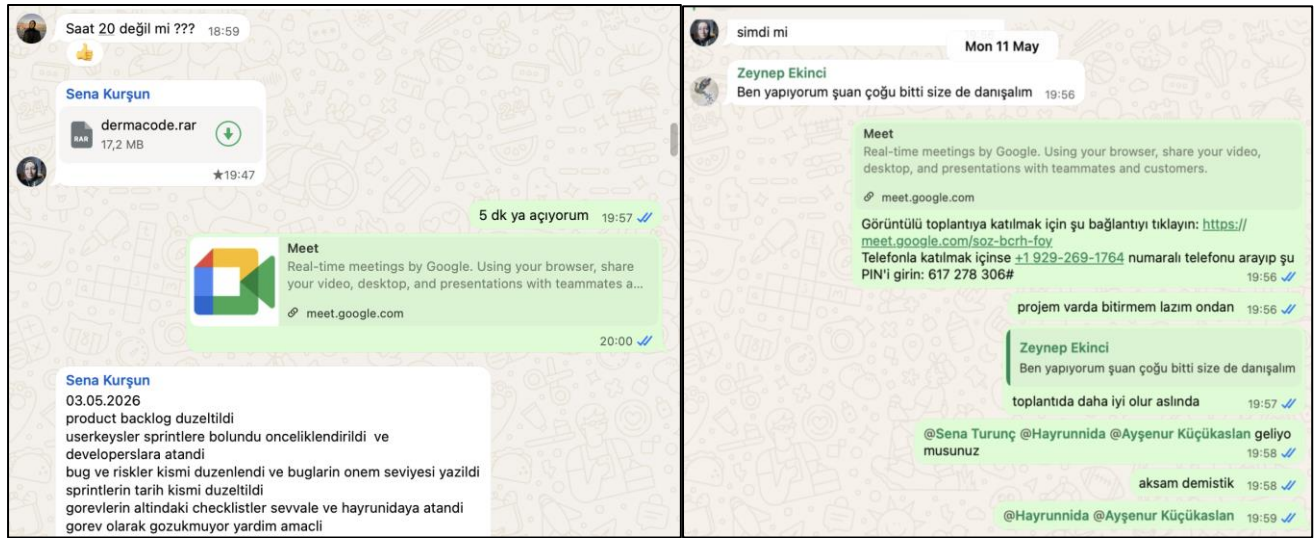
Görsel 3.6 - Frontend Sürüm Kontrol Ağacı (Network Graph): Çoklu ekran tasarımları ve asenkron HTTP entegrasyonları sırasında uygulanan eşzamanlı dallanma hiyerarşisi.

Ek-4 Toplantı Tutanakları

Proje geliştirme süreci boyunca ekip içi koordinasyonu sağlamak ve çevik (agile) adaptasyonu korumak amacıyla düzenli toplantılar gerçekleştirilmiştir. Scrum çerçevesinin zorunlu kıldığı her gün yapılan 15 dakikalık Daily Scrum toplantılarına ek olarak; projenin güncel durumunu ve mimari entegrasyonları detaylı değerlendirmek amacıyla ilk 3 Sprint boyunca 2 günde bir, son Sprint (Sprint 4) aşamasında ise stabilizasyon sağlandığı için 3 günde bir Google Meet üzerinden ekstra senkronizasyon toplantıları yapılmıştır.

Toplantılar zaman yönetimi disiplini gereği saat 20.00'da başlatılmış ve maksimum 30 dakika (Timebox) ile sınırlandırılmıştır. Toplantı gündemleri ve davetleri Scrum Master tarafından WhatsApp ekibi grubu üzerinden önceden duyurulmuştur.

1. Ekip İçi İletişim ve Toplantı Duyuruları



Görsel 4.1 - DermaSmart WhatsApp Geliştirici Grubu: Scrum Master tarafından yapılan Google Meet toplantı duyuruları ve acil kod inceleme (Code Review) çağrılarından örnekler. (Not: Özel konuşmalar kırılarak sadece proje odaklı iletişim eklenmiştir.)

2. Örnek Toplantı Tutanakları (Daily Scrum & Planlama)

Aşağıda, 4 haftalık geliştirme sürecindeki farklı Sprint dönemlerine ait resmi karar tutanaklarından örnekler sunulmuştur:

Toplantı 1 - Sprint 1 Planlama ve Mimari Karar

- **Tarih / Saat:** 28 Nisan 2026 / 20:00 - 20:30
- **Platform:** Google Meet
- **Katılımcılar:** Tüm Ekip (6/6)
- **Gündem ve Alınan Kararlar:**
 - Teknoloji stack (Flutter, ASP.NET Core, SQLite) resmi olarak onaylandı.
 - GitHub reposu açıldı ve ekip üyeleri davet edildi. Feature branch stratejisi kullanılacağı karara bağlandı.
 - Bir sonraki toplantıya kadar veritabanı şemasının taslağının bitirilmesi hedeflendi.

Toplantı 2 - Sprint 2 Daily Scrum (Kriz Yönetimi)

- **Tarih / Saat:** 11 Mayıs 2026 / 20:00 - 20:30
- **Platform:** Google Meet
- **Katılımcılar:** Tüm Ekip (6/6)
- **Gündem ve Alınan Kararlar:**
 - JWT Auth API'lerinin Postman testlerinin başarıyla geçtiği teyit edildi.
 - Kritik Gündem: API testleri sırasında AppUser ve SkinProfile varlıklarının uyuşmadığı ve HTTP 500 hatası alındığı tespit edildi.
 - Karar: Mimari basitleştirilerek tek bir User modeline geçiş yapılmasına ve bu bug çözülene kadar yeni özellik eklenmemesine karar verildi.

Toplantı 3 - Sprint 3 Daily Scrum

- **Tarih / Saat:** 21 Mayıs 2026 / 20:00 - 20:25
- **Platform:** Google Meet
- **Katılımcılar:** Tüm Ekip (6/6)
- **Gündem ve Alınan Kararlar:**
 - Veritabanı HTTP 500 krizi çözüldü, biriken görevler hızla kapatılmaya başlandı.
 - Sabah/Akşam rutin ekranlarının canlı API'ye bağlanması sürecindeki arayüz durum (State) yönetimi tartışıldı.
 - Filtreleme algoritmasının başarıyla çalıştığı mobil arayüz üzerinden test edildi.

Toplantı 4 - Sprint 4 Senkronizasyon (Dağıtım Planı)

- **Tarih / Saat:** 1 Haziran 2026 / 20:00 - 20:20
- **Platform:** Google Meet
- **Katılımcılar:** Tüm Ekip (6/6)
- **Gündem ve Alınan Kararlar:**
 - Çapraz platform (iOS/Android) E2E testlerinin sorunsuz tamamlandığı onaylandı.
 - Bulut sunucu maliyet iptali sebebiyle, jüri sunumu için tam izole yerel ağ (Local IP) stratejisine geçiş yapıldı.
 - Sistemin canlı sürüme alındığı ve geliştirmenin dondurulduğu ilan edildi.

Ek-5 Kullanıcı Kılavuzu

Bu kılavuz, DermaSmart mobil uygulamasının son kullanıcı tarafından ilk kurulumdan, günlük rutin takibine kadar nasıl kullanılacağını adım adım açıklamaktadır. Uygulama arayüzü, karmaşadan uzak ve yönlendirici bir tasarıma sahiptir.

Adım 1: Karşılama ve Cilt Tipi Belirleme (Onboarding & Anketi)

1. Uygulamayı ilk kez açtığınızda karşınıza login ve register sayfaları çıkıyor.
2. Register sayfasından hemen sonra karşınıza adım tabanlı (stepper) sorular çıkacaktır. Cildinizin gün içindeki yağlanma durumu, gözenek yapısı ve hassasiyet derecesi gibi soruları yanıtlayarak ilerleyin.
3. Anket sonunda sistem, verdiğiniz yanıtları arka planda işleyerek dermatolojik profilinizi (Örn: *Karma ve Akneye Meyilli*) oluşturur ve ana ekrana yönlendirilirsiniz.

Adım 2: Dinamik Katalog, Arama ve Filtreleme

1. Ana ekrandan alt gezinme çubuğundaki "**Katalog**" sekmesine geçtiğinizde, sistem veri tabanındaki ürünleri varsayılan olarak iki kolonlu kart formatında ekrana getirir.
2. Belirli bir kozmetiği bulmak için en üstte yer alan "**Ürün veya marka ara...**" çubuğuna dokunarak metin girişi yapabilirsiniz.
3. Ürün havuzunu daraltmak için arama çubuğunun hemen altında yatay olarak sıralanmış **Cilt Tipi Filtre Etiketlerinden** (*Tümü, Yağlı Cilt, Kuru Cilt, Karma Cilt*) cildinize uygun olanına dokununuz.
4. Yaptığınız filtreleme veya arama sonucunda veri tabanından dönen eşleşme sayısı, listenin hemen üstündeki sayaçta (Örn: "*9 ürün bulundu*") dinamik olarak güncellenir.
5. Listelenen ürün kartlarının üzerinde yer alan açık mavi etiketler (Örn: *Kuru, Yağlı, Karma, Hassas*), o ürünün formülasyonunun hangi cilt tipleriyle uyumlu olduğunu son kullanıcıya şeffafça gösterir.

Adım 3: Ürün Analizi ve Çakışma Kontrolü

1. İncelemek istediğiniz ürünün üzerine dokunarak "Ürün Detay Sayfası"na girin.
2. Bu ekranda, sistemin veri tabanından çektiği "İçerik Analiz Kartı"nı inceleyin. Ürünün içindeki aktif ve yardımcı maddeler cilt tipinize göre yeşil (faydalı) veya sarı (dikkatli kullanım) etiketlerle listelenir.
3. Ürünü planınıza dahil etmek için "**Rutine Ekle**" butonuna basın. Uygulama, arka planda çalışan Kural Motoru sayesinde bu ürünü, mevcut rutininizdeki diğer ürünlerle çapraz kontrole sokar. Eğer bir kimyasal çakışma (Örn: Peeling asidi ile Retinol uyumsuzluğu) tespit edilirse, sistem sizi kırmızı bir uyarı ile bilgilendirir ve cildinizi korumak adına ekleme işlemi engeller.

Adım 4: Sabah (AM) ve Akşam (PM) Rutin Takibi

1. Çakışma testini başarıyla geçen ürünleri eklerken karşınıza çıkan ekrandan "**Sabah Rutini (AM)**" veya "**Akşam Rutini (PM)**" seçiminizi yapın.
2. Alt gezinme çubuğundan "**Rutinlerim**" sekmesine geçin.
3. Bu ekranda, sabah ve akşam kullanmanız gereken ürünleri doğru kullanım sırasına göre (Örn: Temizleyici -> Tonik -> Serum -> Nemlendirici) dizilmiş halde görebilirsiniz.
4. Kullandığınız ürünlerin yanındaki onay kutucuklarını işaretleyerek günlük cilt bakım takibinizi tamamlayın.

Adım 5: Gelişim Takibi ve İstatistikler (İlerleme Ekranı)

1. Günlük cilt bakım rutininizi ne ölçüde istikrarlı sürdürdüğünüzü görmek için alt gezinme çubuğundaki "**İlerleme**" sekmesine dokununuz.
2. Bu ekran, kullanıcı motivasyonunu artırmak amacıyla oyunlaştırma (gamification) dinamikleriyle tasarlanmıştır. Ekranın en üstünde yer alan özet kartında, rutininizi peş peşe kaç gün bozmadan tamamladığınızı gösteren "**Günlük Seri (Streak)**" bilginiz yer alır.
3. Orta bölümde konumlandırılan istatistik panelleri üzerinden; mevcut "Gün Seri"nizi, rutini tamamladığınız "Toplam Gün" sayısını ve bulunduğunuz ay içerisindeki rutin tamamlama istikrarınızı ("**% Bu Ay**") oransal olarak takip edebilirsiniz.

4. Ekranın alt kısmında yer alan "**Rutin Takvimi**" üzerinden geçmişe dönük verilerinizi inceleyebilirsiniz. Günlük rutin görevlerinin (AM/PM) başarıyla tamamlandığı günler, takvim matrisi üzerinde **yeşil** renkle vurgulanarak size aylık bazda görsel bir performans raporu sunar.
5. Verilerinizin sunucu ile son durumunu eşitlemek isterseniz sağ üst köşedeki "**Yenile**" (**Refresh**) ikonuna dokunarak istatistiklerinizi anlık olarak güncelleyebilirsiniz.